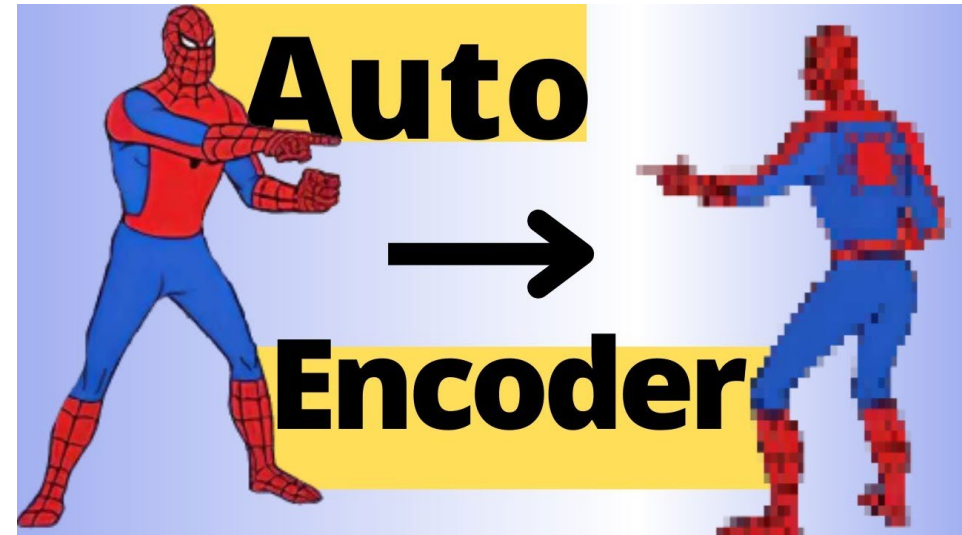


~~DSK809 Deep Learning:~~ Autoencoders

Anton D. Lautrup
lautrup@imada.sdu.dk



Overview

01

Autoencoders

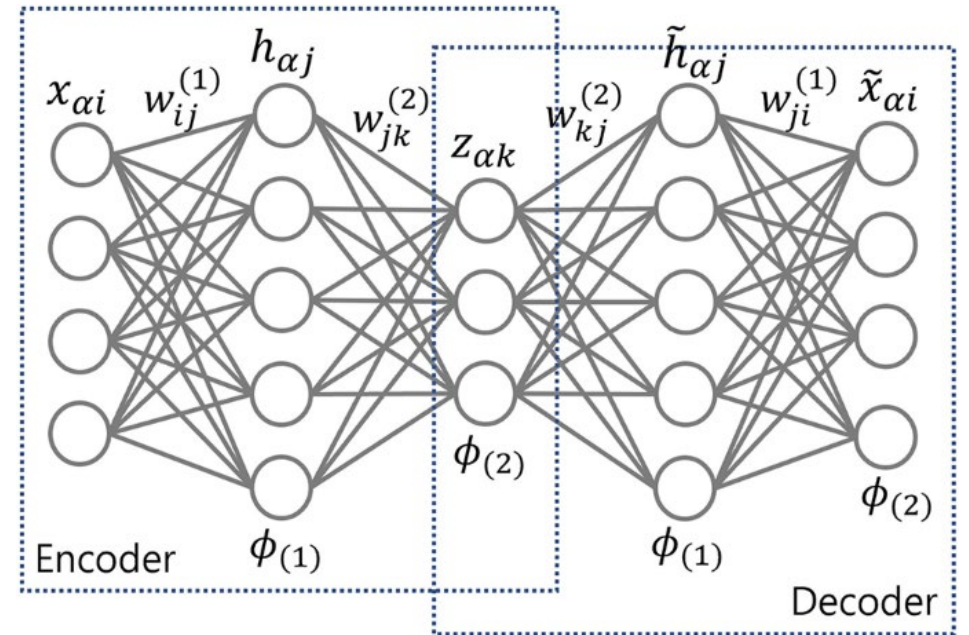
02

Latent space

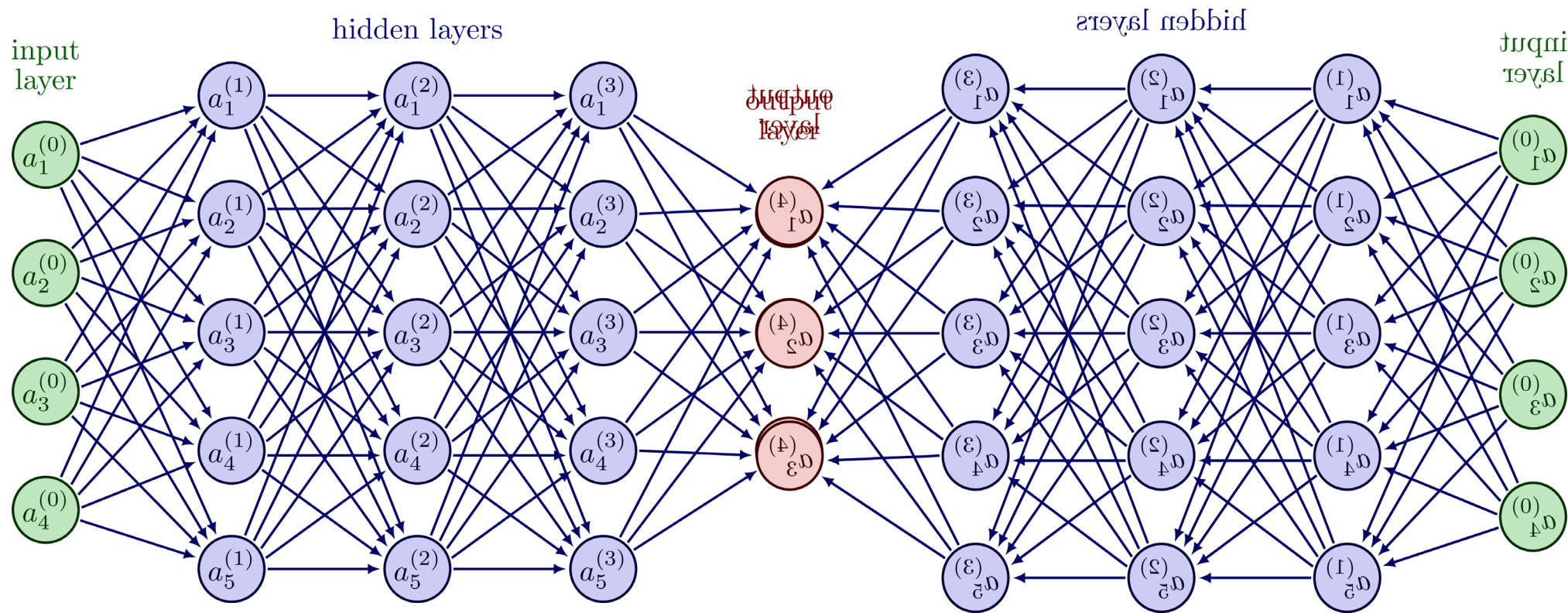
03

Variational
autoencoders

Autoencoders



What are autoencoders really?

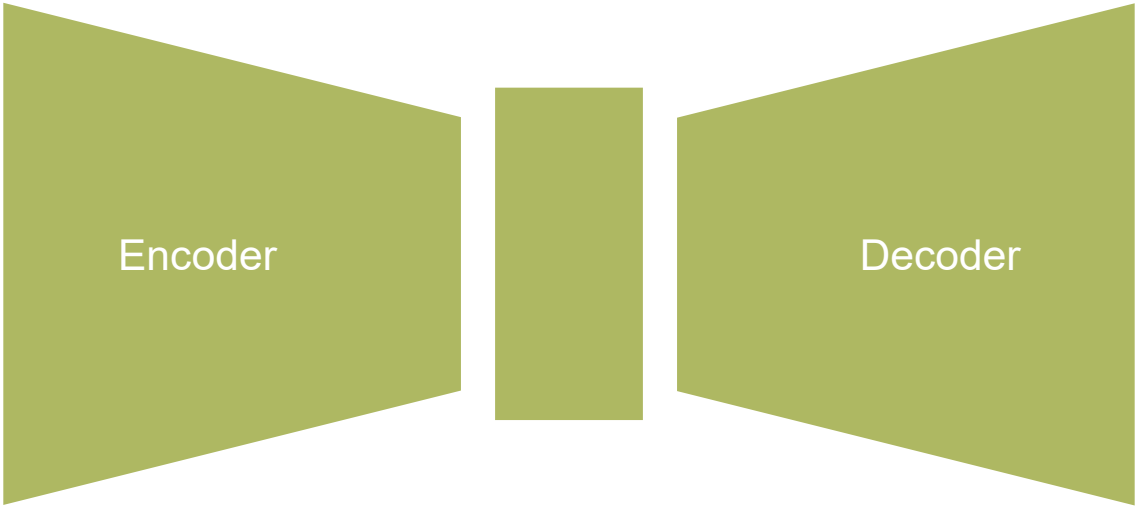


What is the fuss about?



ID	Height	Age	BP	Colour	Pet
1	180	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps ...



‘Bottleneck’



ID	Height	Age	BP	Colour	Pet
1	180	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps ...

What is the use?



Department of Mathematics
and Computer Science

What is the fuss about?



ID	Height	Age	BP	Colour	Pet
1	180	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps ...

Encoder

Bottleneck

Decoder



ID	Height	Age	BP	Colour	Pet
1	180	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps ...

SDU

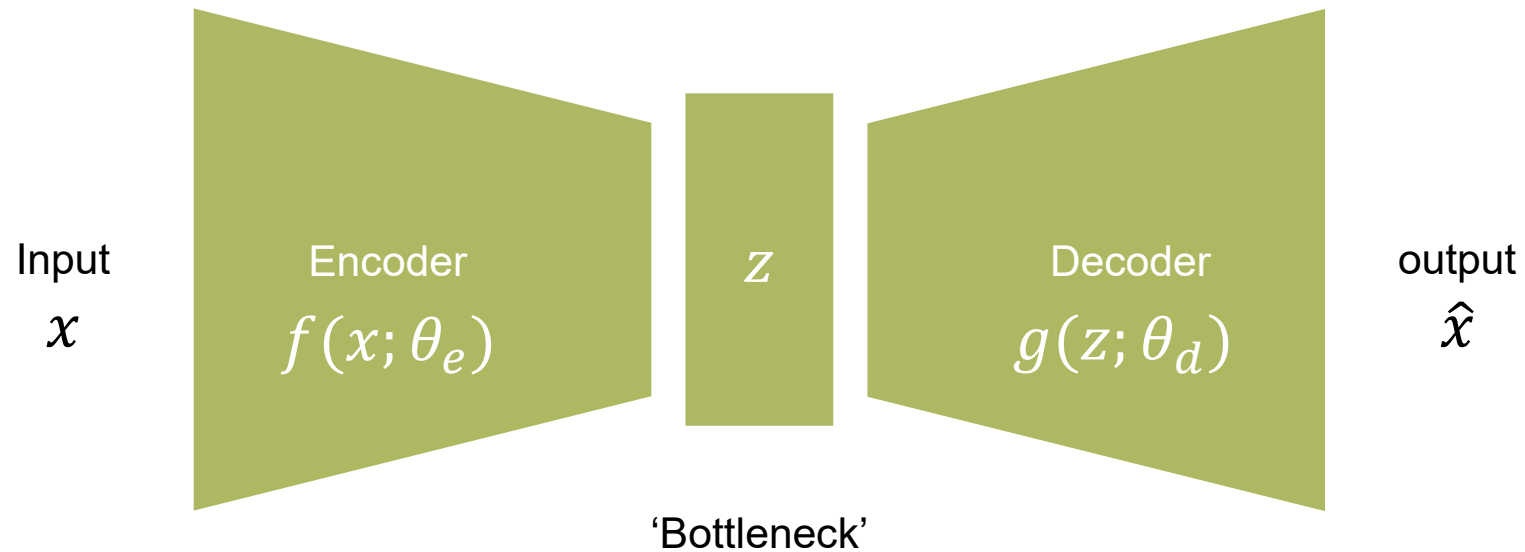
Anton D. Lautrup ~ lautrup@imada.sdu.dk

23 October 2025

sdu.dk

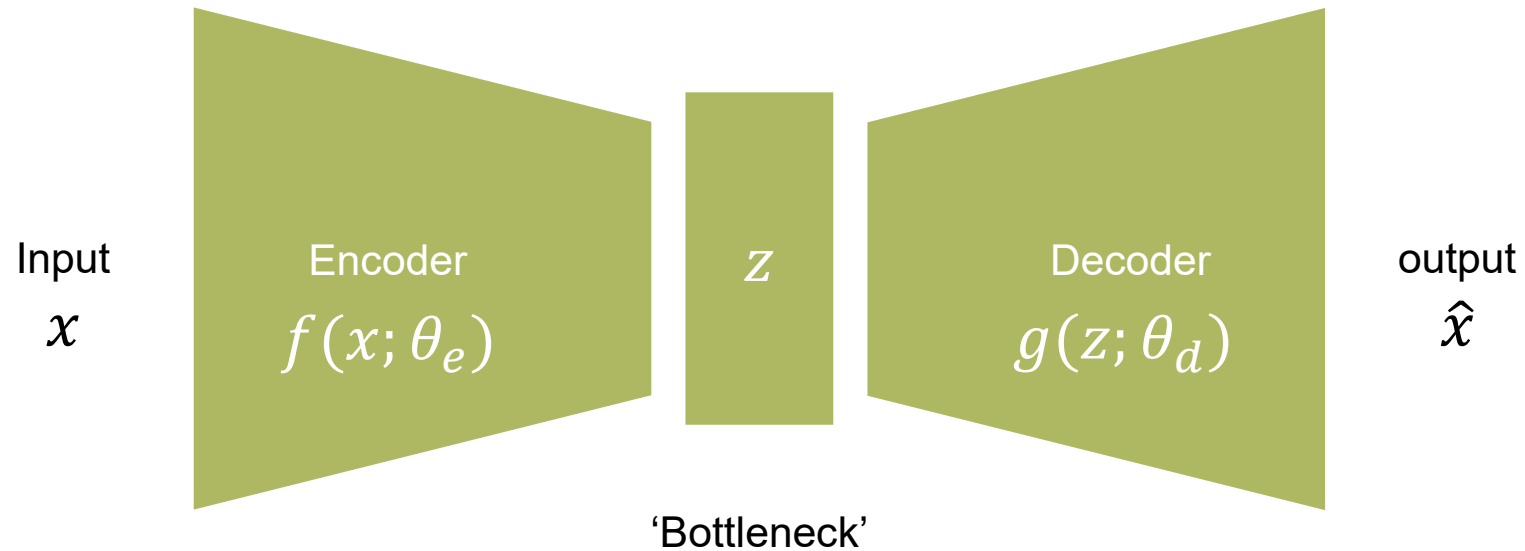
#sdudk

Objective function



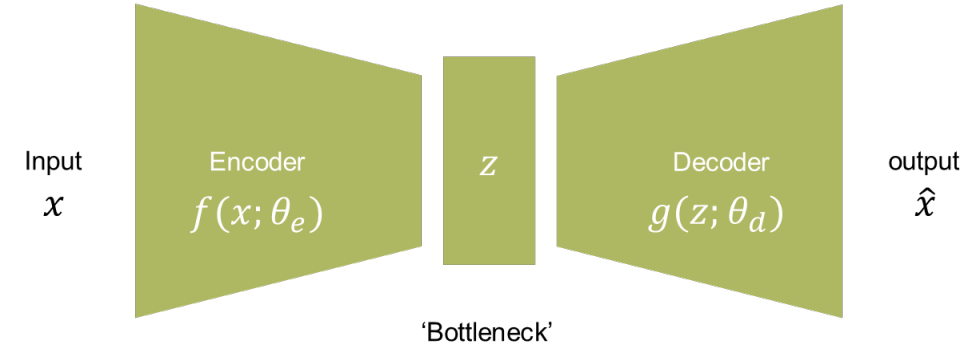
Find optimal parameters $\theta = \{\theta_e, \theta_d\}$ such that input and reconstruction are equal

Objective function



$$\operatorname{argmin}_{\theta_e, \theta_d} \mathcal{L}(x_i, g(f(x_i; \theta_e); \theta_d))$$

Reconstruction Loss



$$\min_{\theta_e, \theta_d} \mathcal{L}(x_i, g(f(x_i; \theta_e); \theta_d))$$

	$\mathcal{L}(x, \hat{x})$
Mean squared error	$\frac{1}{N} \sum_i (x_i - \hat{x}_i)^2$
Binary cross entropy	$-\frac{1}{N} \sum_i [x_i \log \hat{x}_i - (1 - x_i) \log(1 - \hat{x}_i)]$
Mean absolute error	$\frac{1}{N} \sum_i x_i - \hat{x}_i $

Standard choice: continuous data

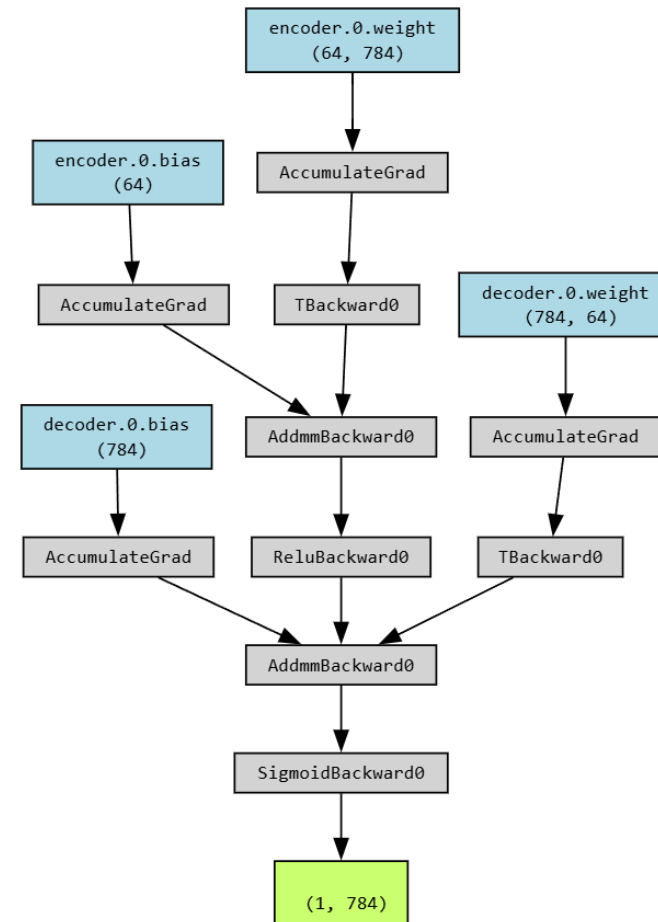
Normalised $[0,1]$ data + sigmoid

Unusual, less sensitive to outliers

Building autoencoder in PyTorch

```
class Autoencoder(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_size, latent_size),
            nn.ReLU(True)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_size, input_size),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x
```



Minimal training loop

```
### Example use
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = Autoencoder(28*28, 64).to(device)

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

dataloader = torch.utils.data.DataLoader(training_data, batch_size=64, shuffle=True)
for epoch in range(5):
    for data, _ in dataloader:
        data = data.view(data.size(0), -1).to(device) # flatten
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, data)
        loss.backward()
        optimizer.step()

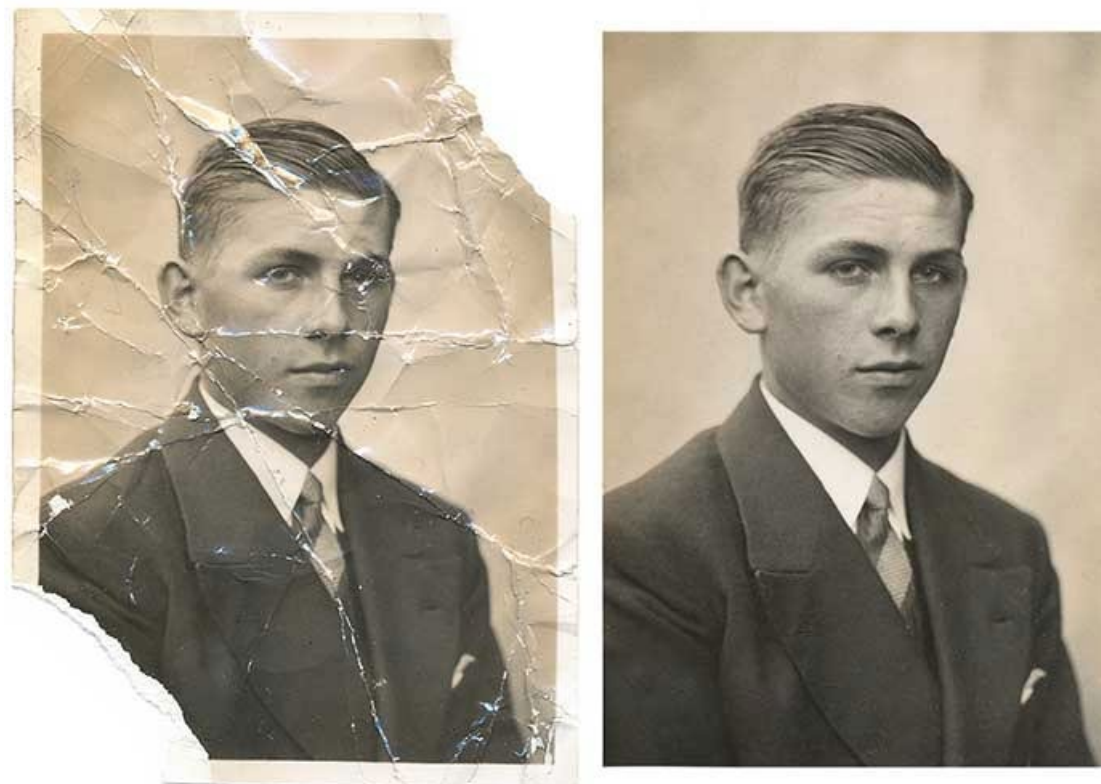
    print(f"Epoch {epoch+1}, Loss: {loss.item()}")
```

Building autoencoder in PyTorch

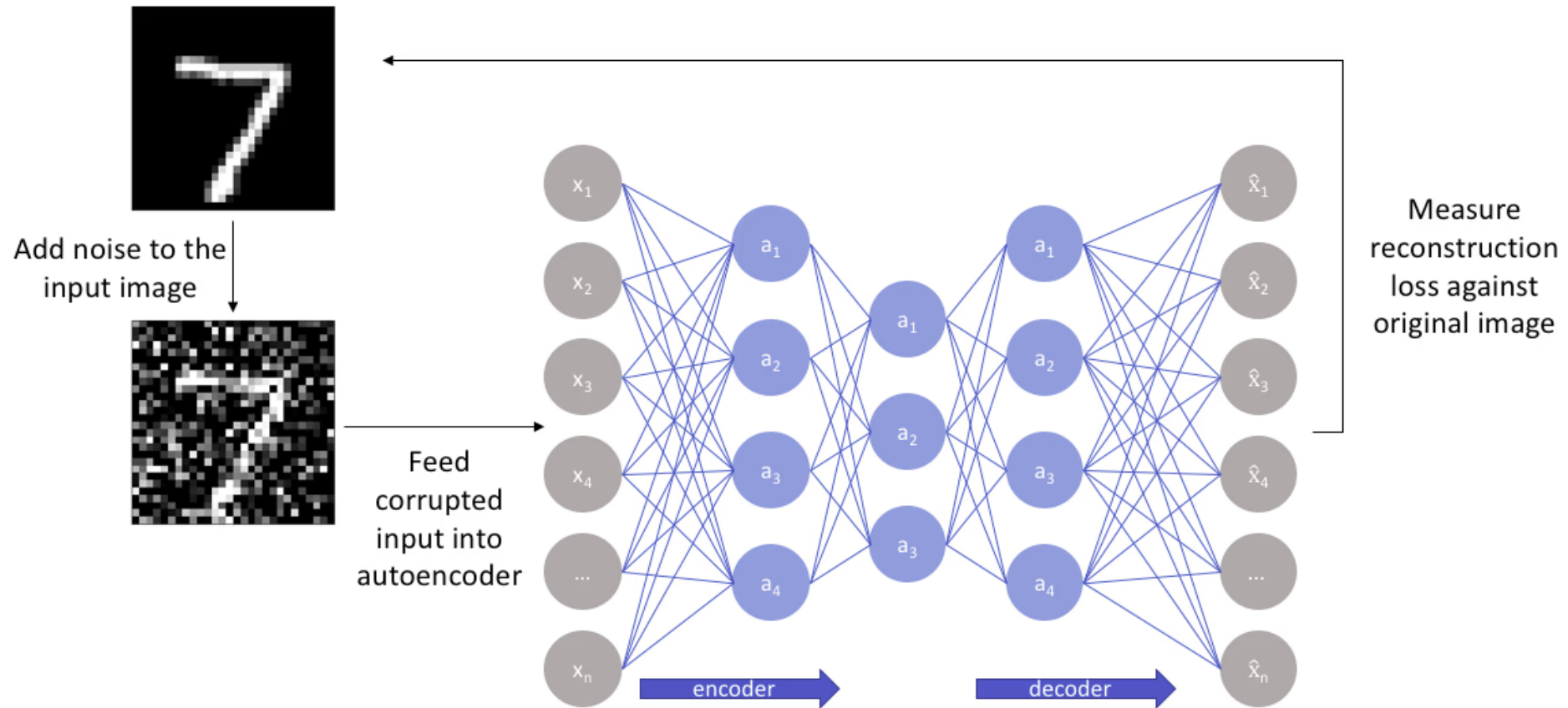
```
Autoencoder(  
    (encoder): Encoder(  
        (encoder): Sequential(  
          (0): Linear(in_features=784, out_features=64, bias=True)  
          (1): ReLU(inplace=True)  
        )  
      )  
    (decoder): Decoder(  
        (decoder): Sequential(  
          (0): Linear(in_features=64, out_features=784, bias=True)  
          (1): Sigmoid()  
        )  
      )  
  )  
)
```

```
class Decoder(torch.nn.Module):  
    def __init__(self, input_size, latent_size):  
        super(Decoder, self).__init__()  
        self.decoder = nn.Sequential(  
            nn.Linear(latent_size, input_size),  
            nn.Sigmoid()  
        )  
  
    def forward(self, x):  
        x = self.decoder(x)  
        return x  
  
class Encoder(torch.nn.Module):  
    def __init__(self, input_size, latent_size):  
        super(Encoder, self).__init__()  
        self.encoder = nn.Sequential(  
            nn.Linear(input_size, latent_size),  
            nn.ReLU(True)  
        )  
  
    def forward(self, x):  
        x = self.encoder(x)  
        return x  
  
class Autoencoder(torch.nn.Module):  
    def __init__(self, input_size, latent_size):  
        super(Autoencoder, self).__init__()  
        self.encoder = Encoder(input_size, latent_size)  
        self.decoder = Decoder(input_size, latent_size)  
  
    def forward(self, x):  
        x = self.encoder(x)  
        x = self.decoder(x)  
        return x
```

Case example: Image Restoration



Denoising Autoencoder

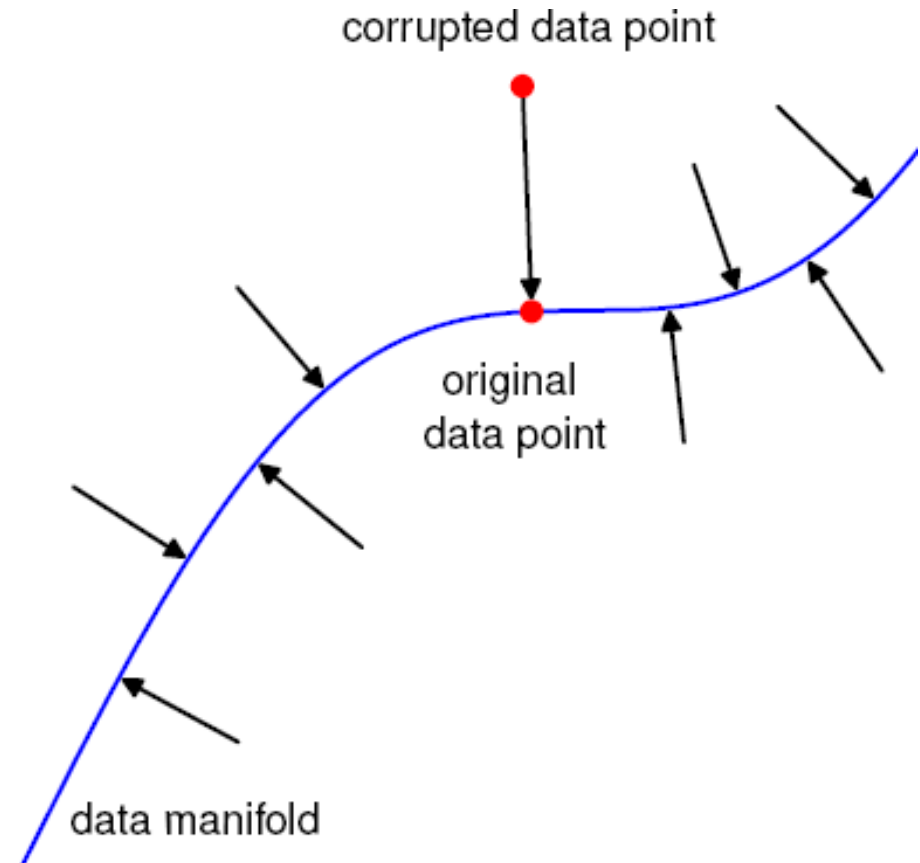


Denoising Autoencoder

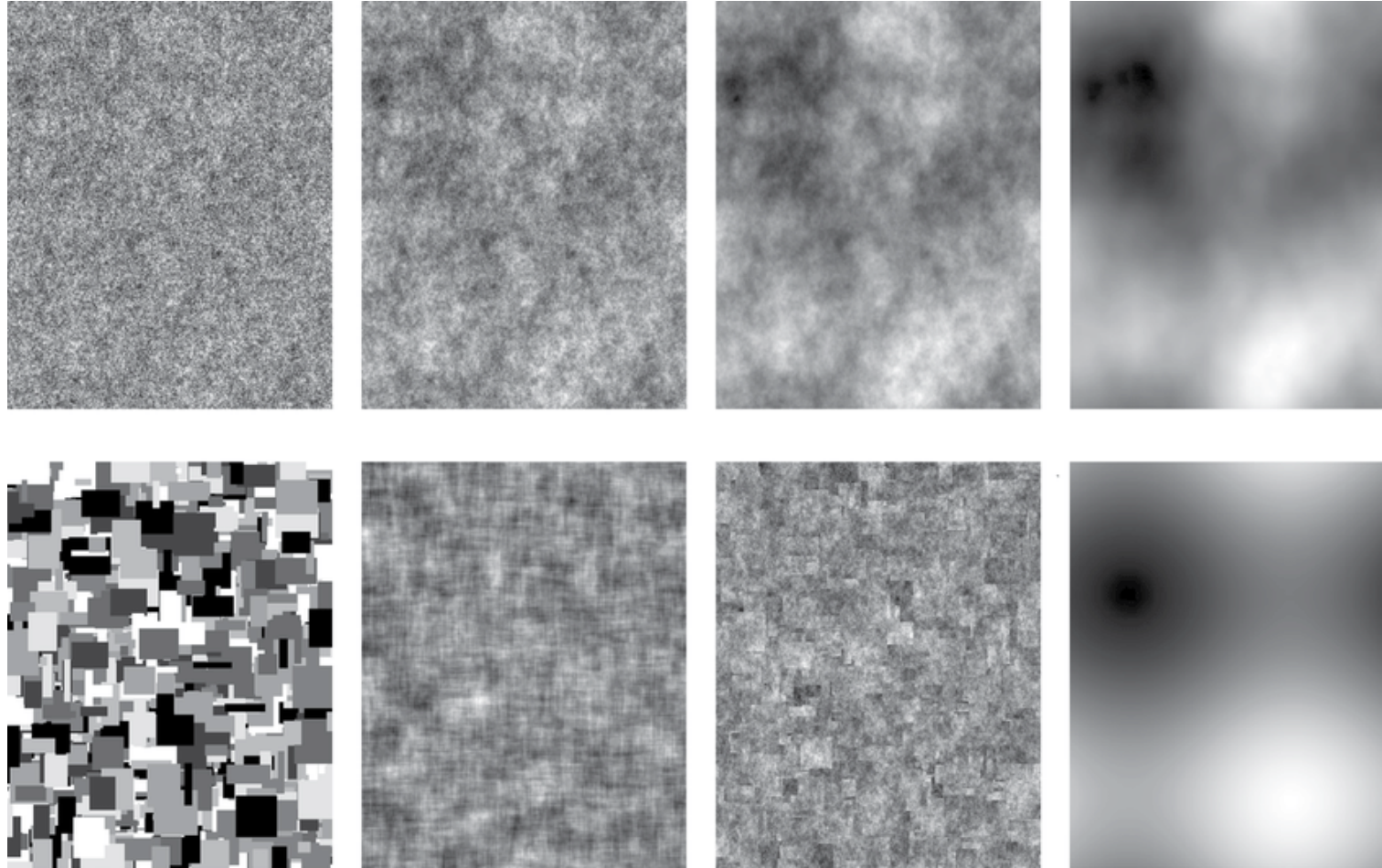
Minimise:

$$\mathcal{L}_{rec} = \frac{1}{2} |g(f(\tilde{x})) - x|^2$$

Where $\tilde{x}$ is a corrupted version of x .



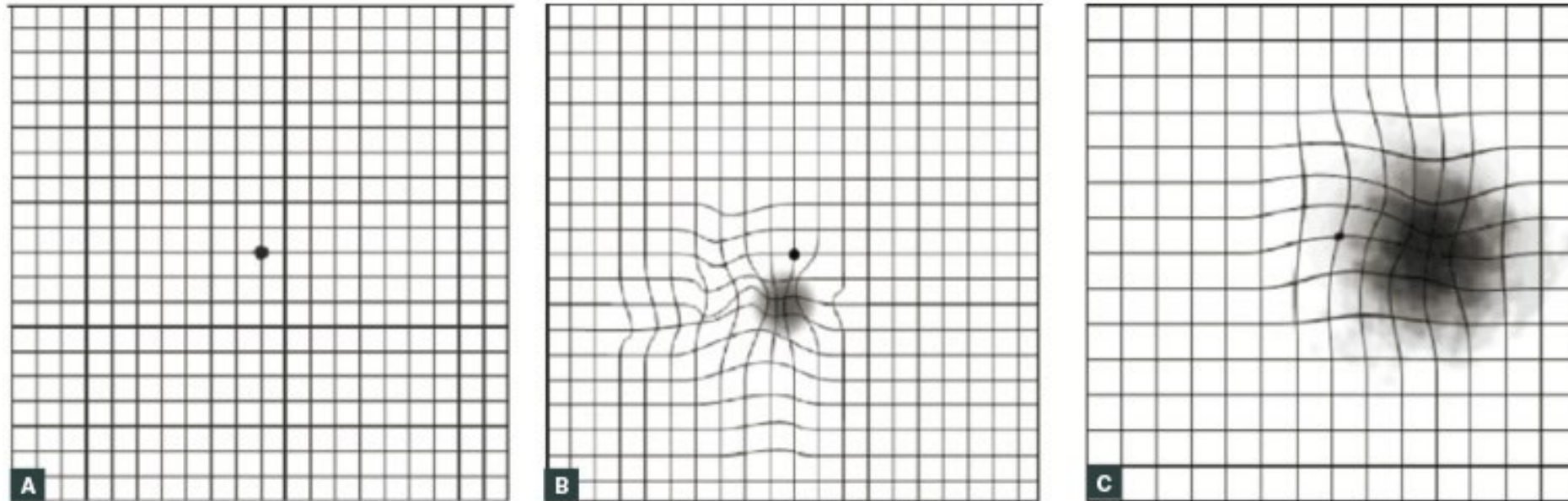
Filters



Artefacts



Distortion

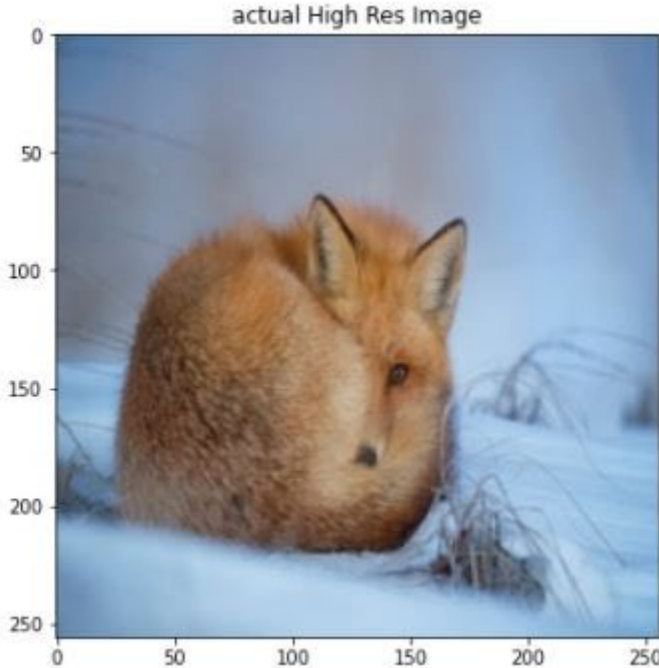
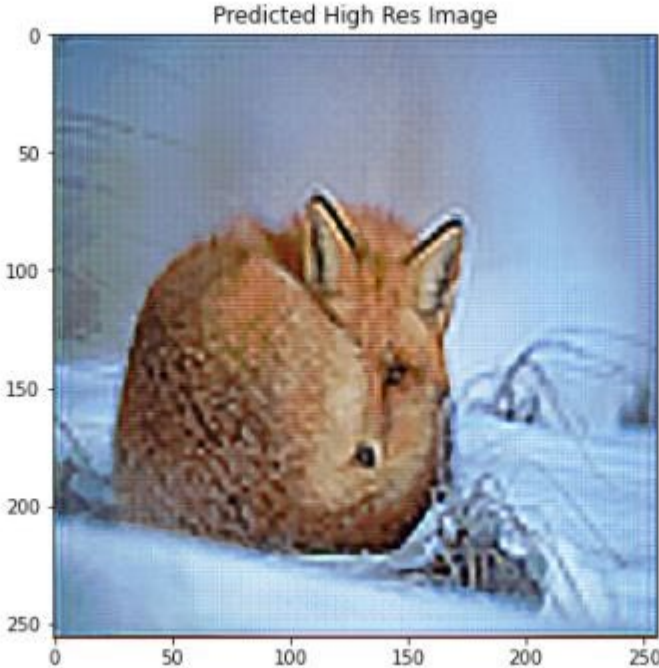
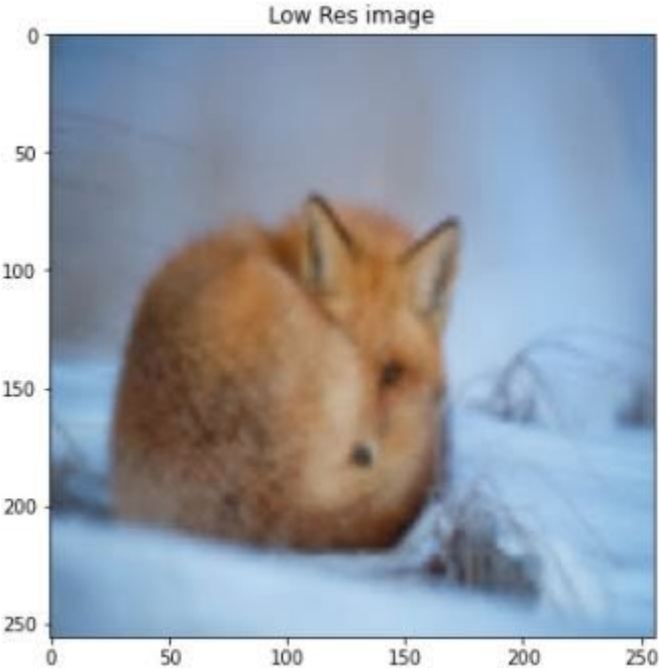


Masking



C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_19

Compression





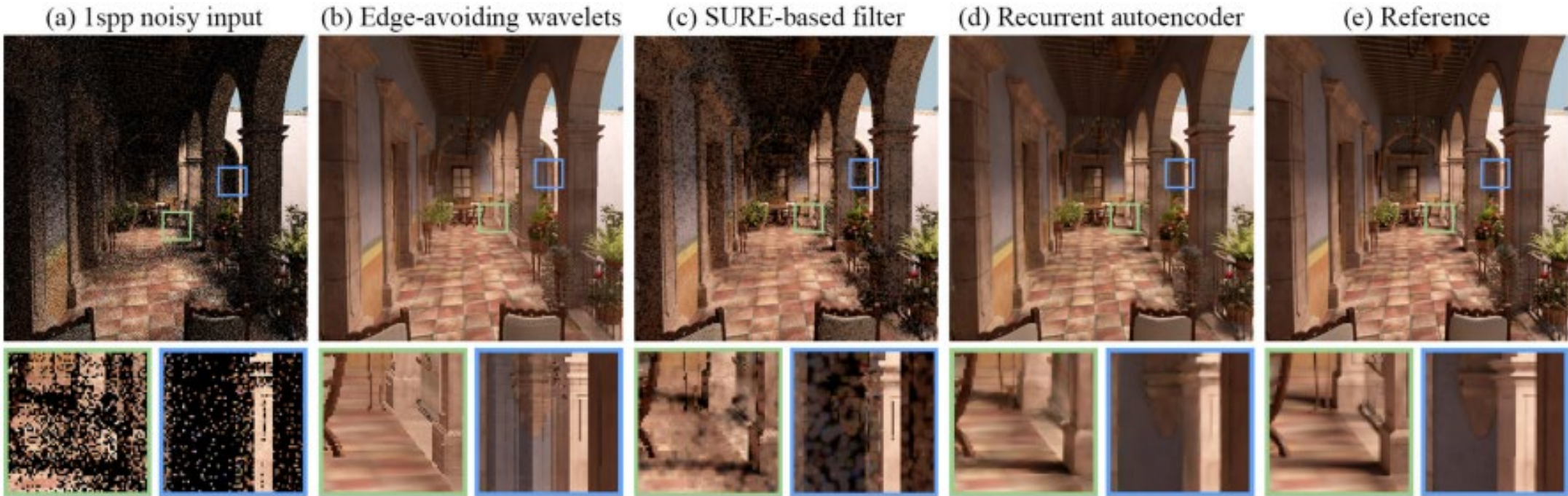
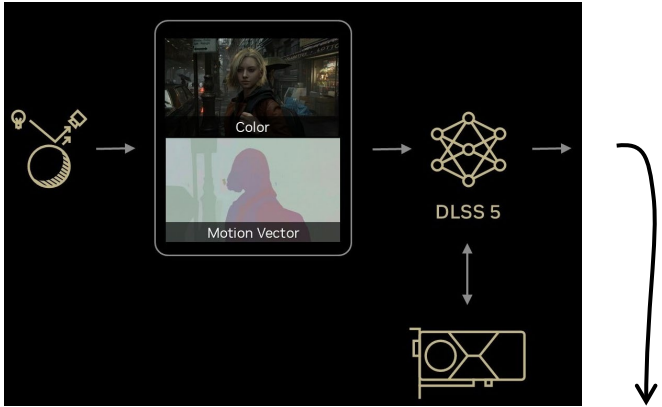
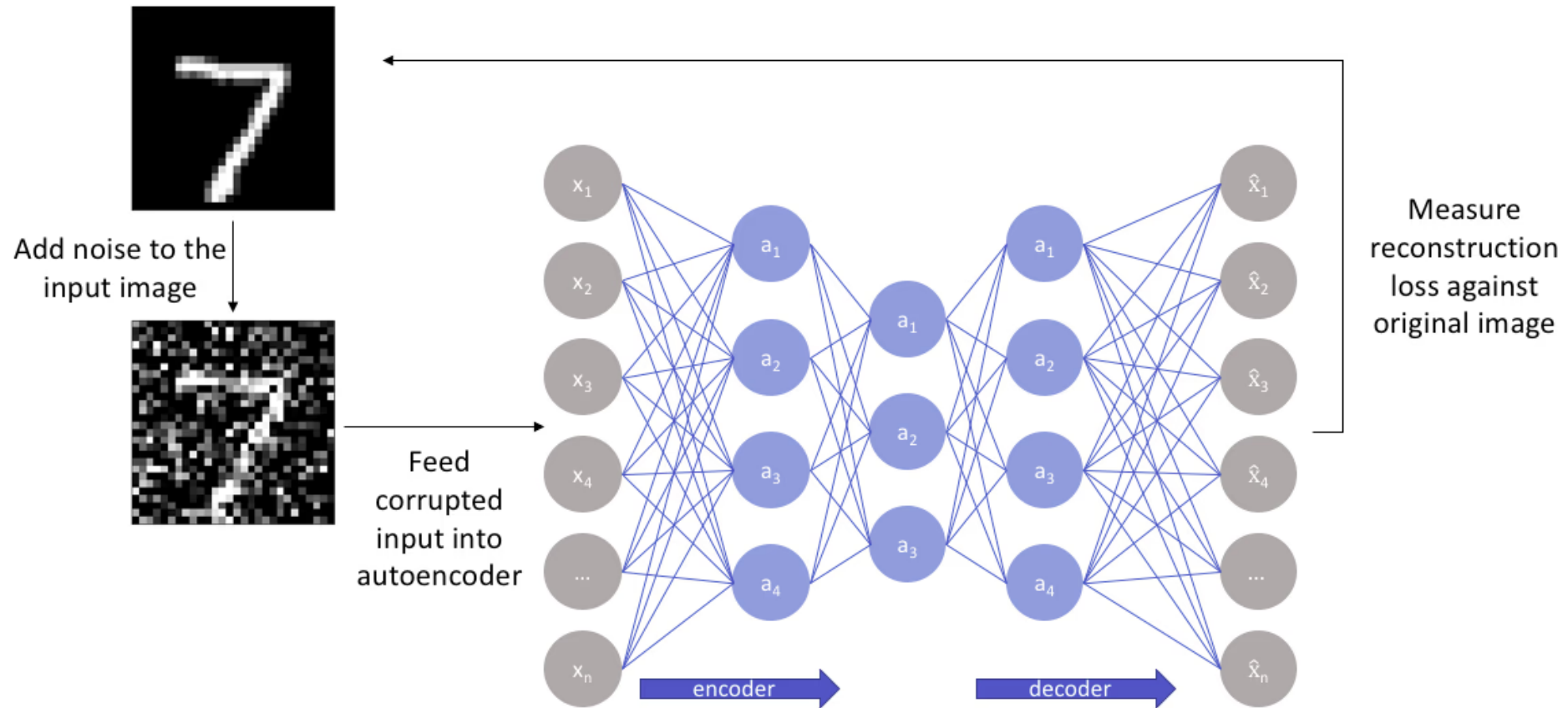


Fig. 1. Left to right: (a) noisy image generated using path-traced global illumination with one indirect inter-reflection and 1 sample/pixel; (b) edge-avoiding wavelet filter [Dammertz et al. 2010] (10.3ms at 720p, SSIM: 0.7737); (c) SURE-based filter [Li et al. 2012] (74.2ms, SSIM: 0.5960); (d) our recurrent denoising autoencoder (54.9ms, SSIM: 0.8438); (e) reference path-traced image with 4096 samples/pixel.

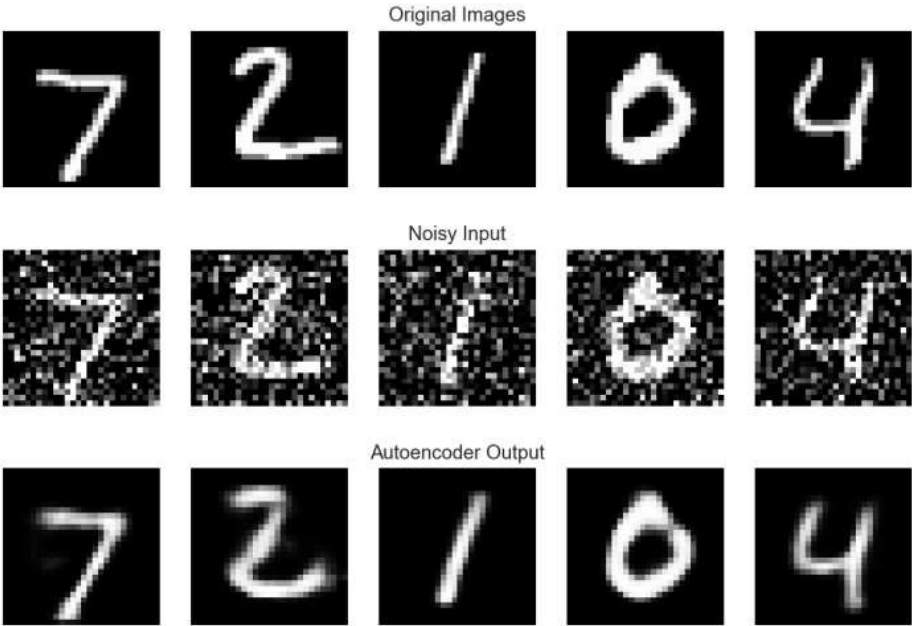
DLSS 5



Denoising Autoencoder



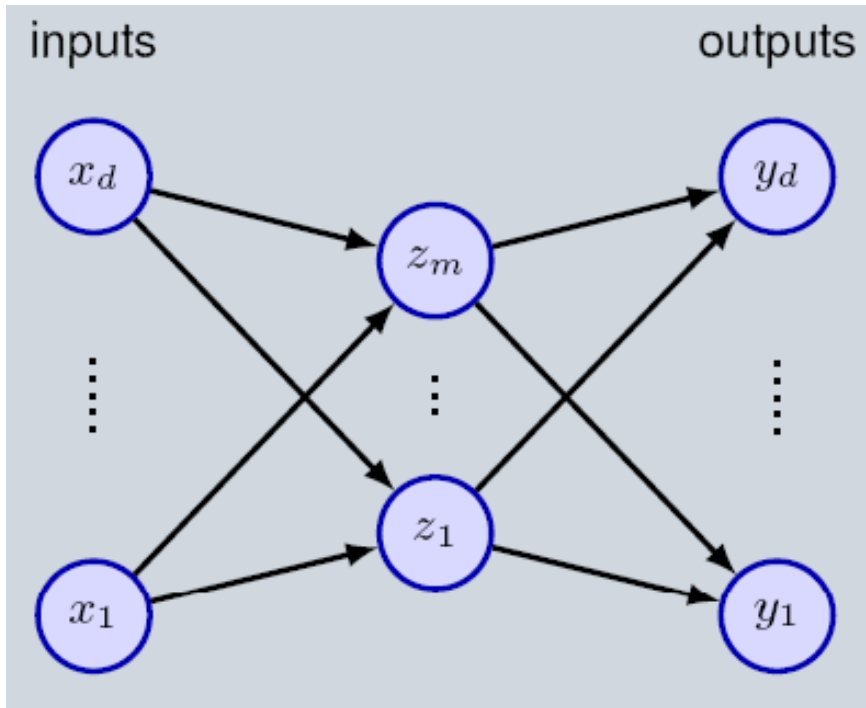
Generalisation?



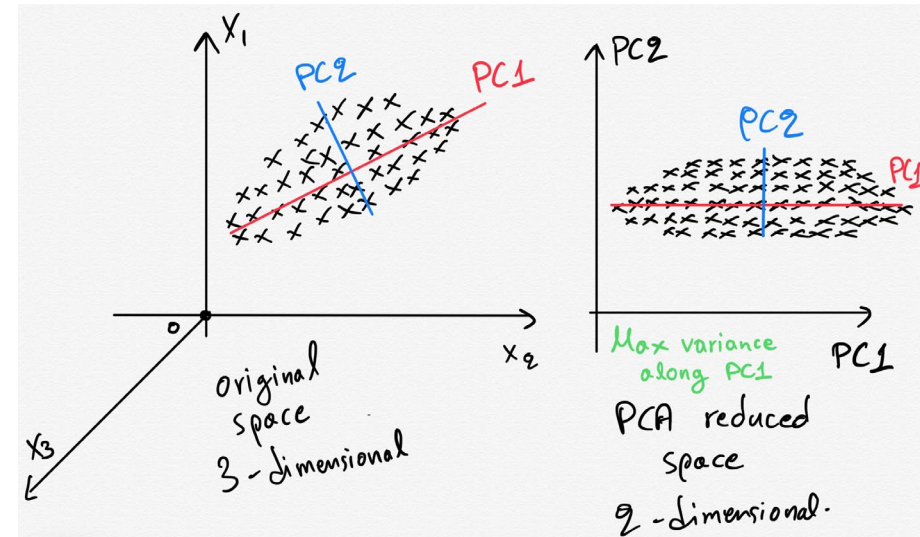
Latent space



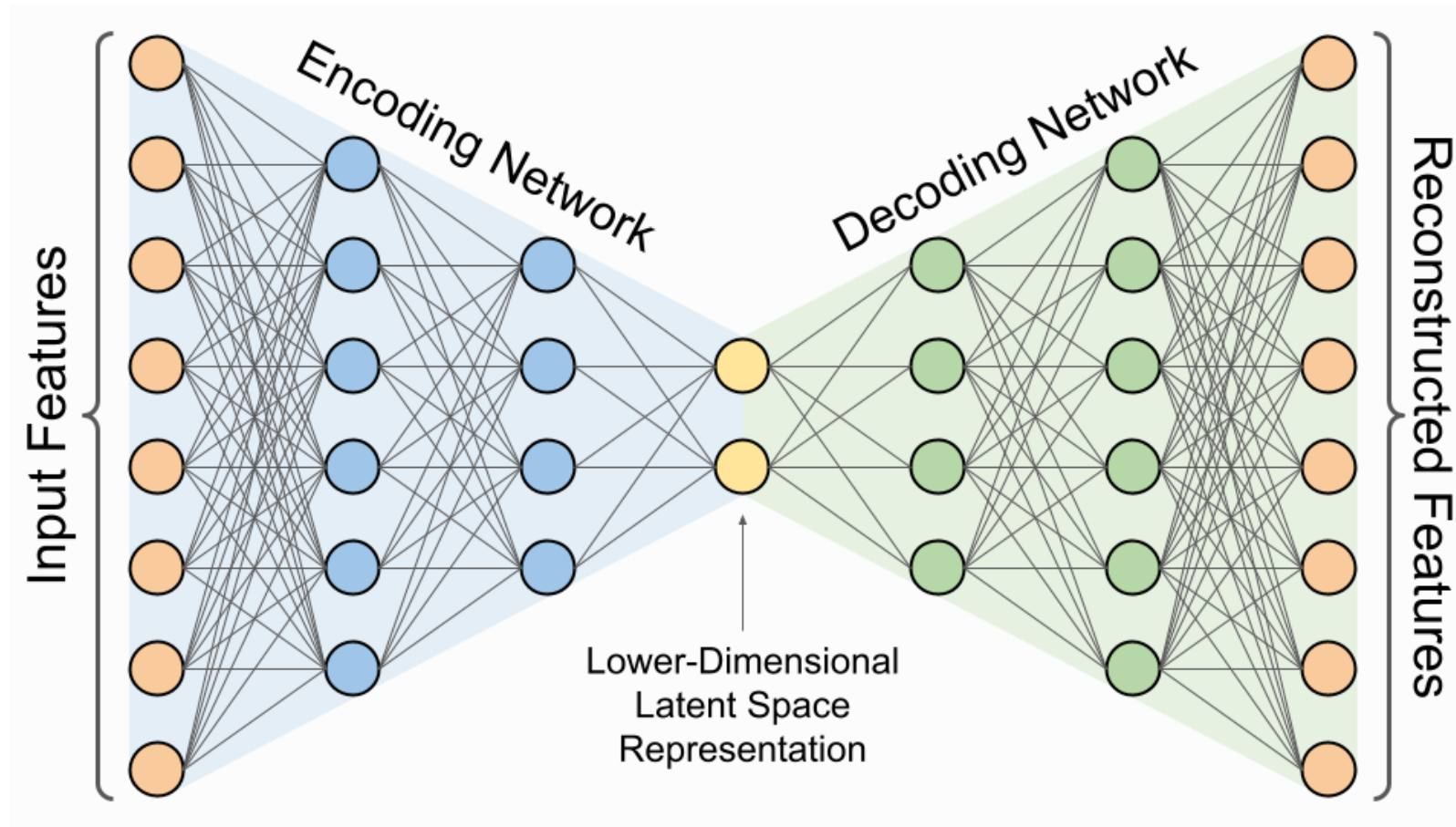
Edge case with no non-linearities



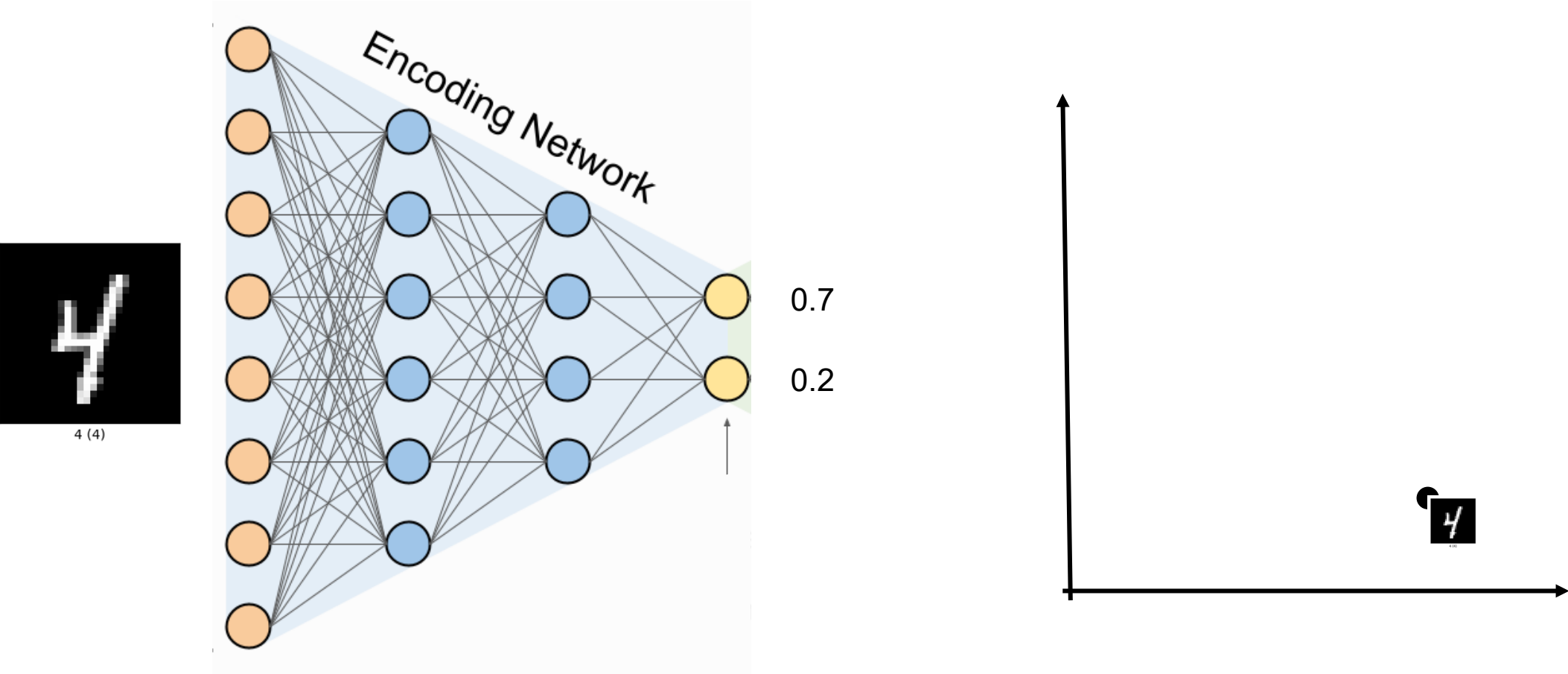
- Both encoder f and decoder g are just linear projections



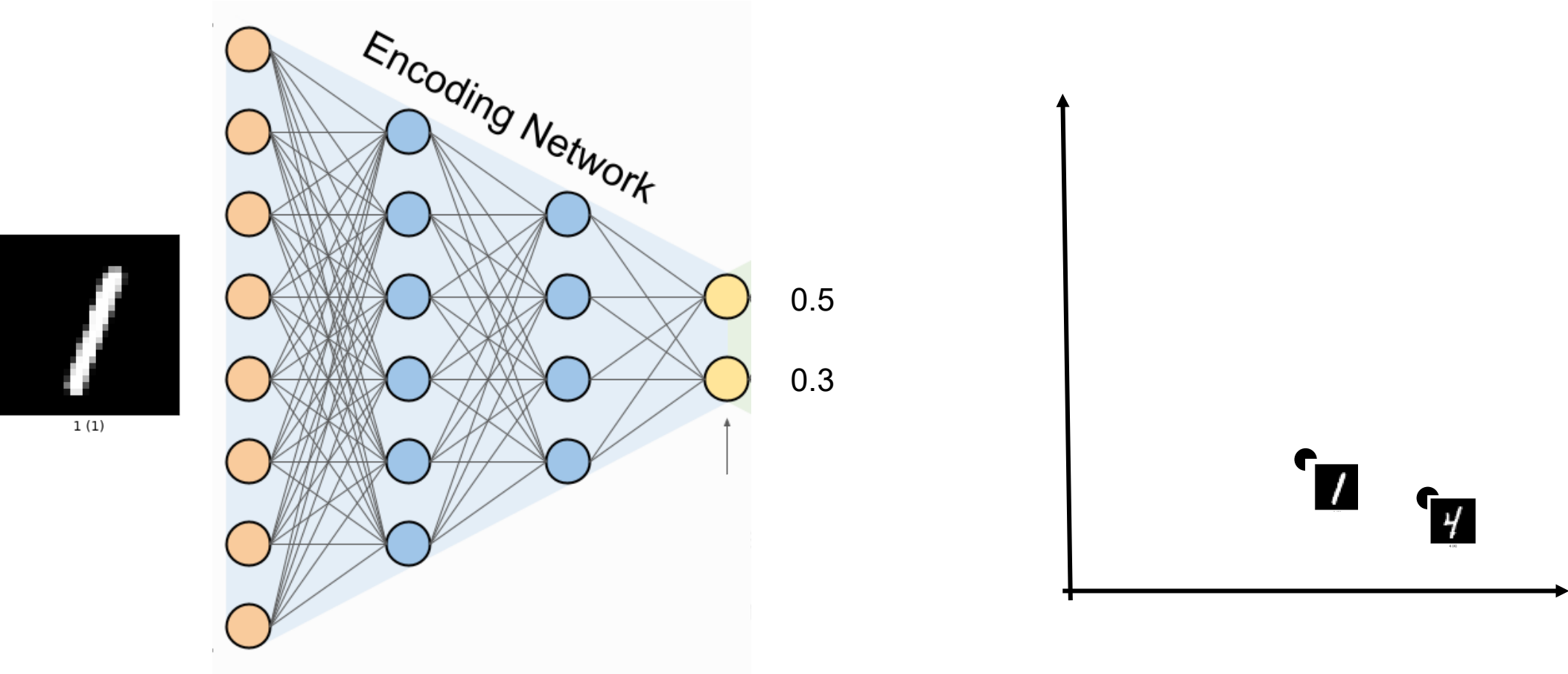
Automatic feature extraction



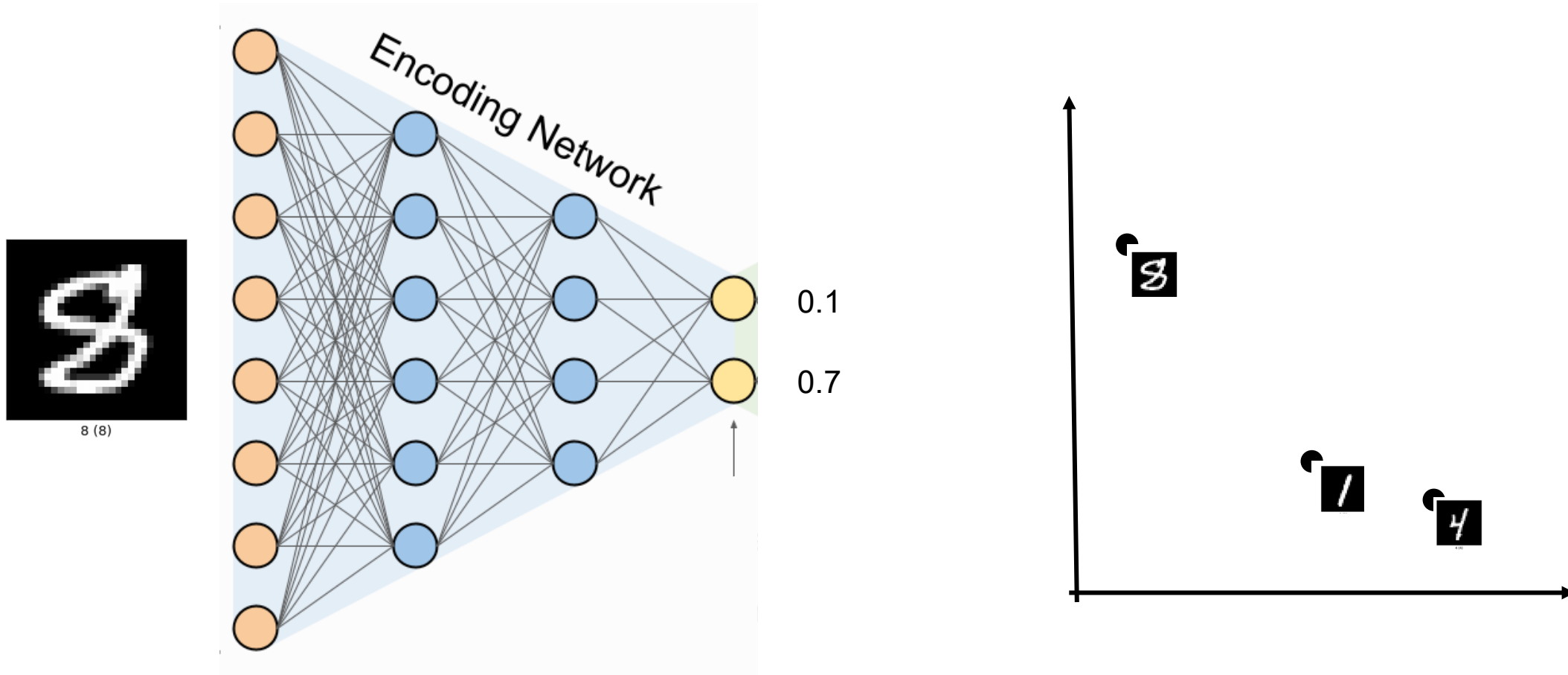
Low dimensional representation



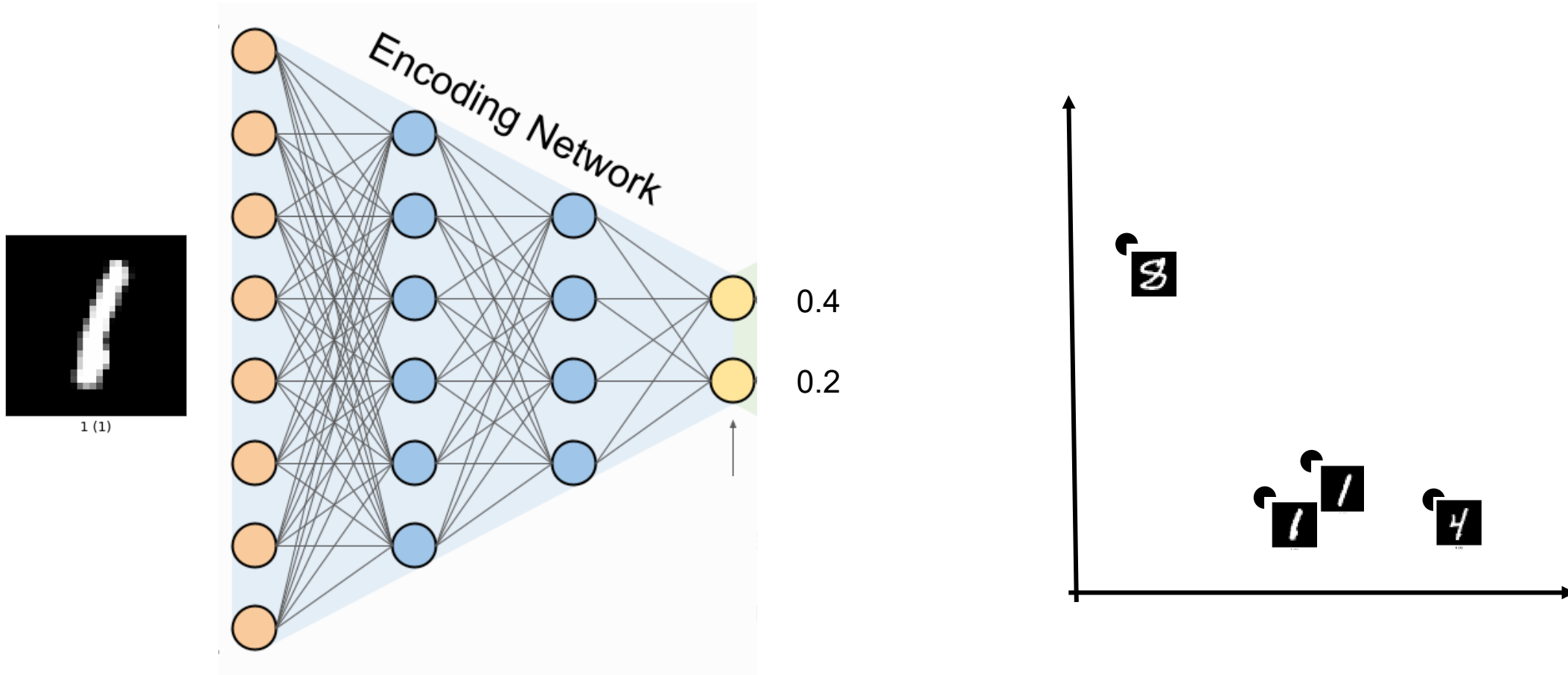
Low dimensional representation



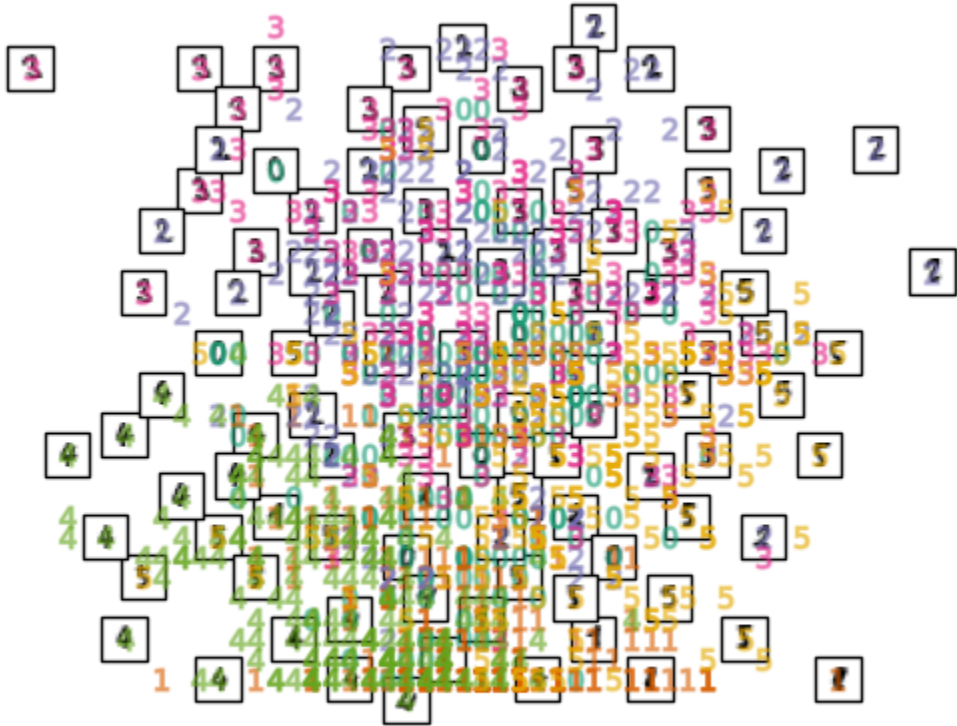
Low dimensional representation



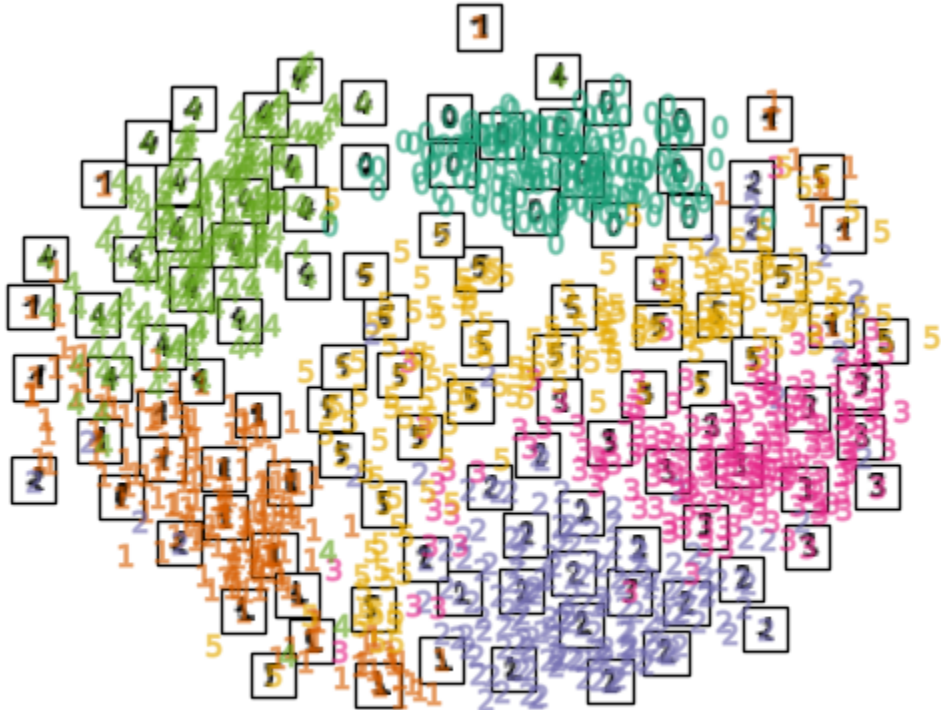
Low dimensional representation



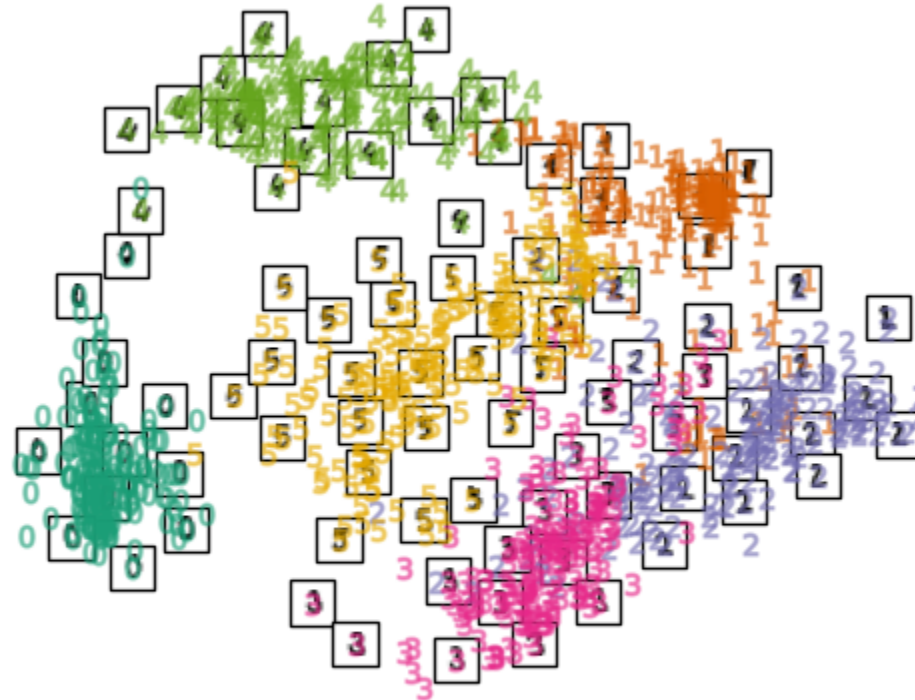
Variation



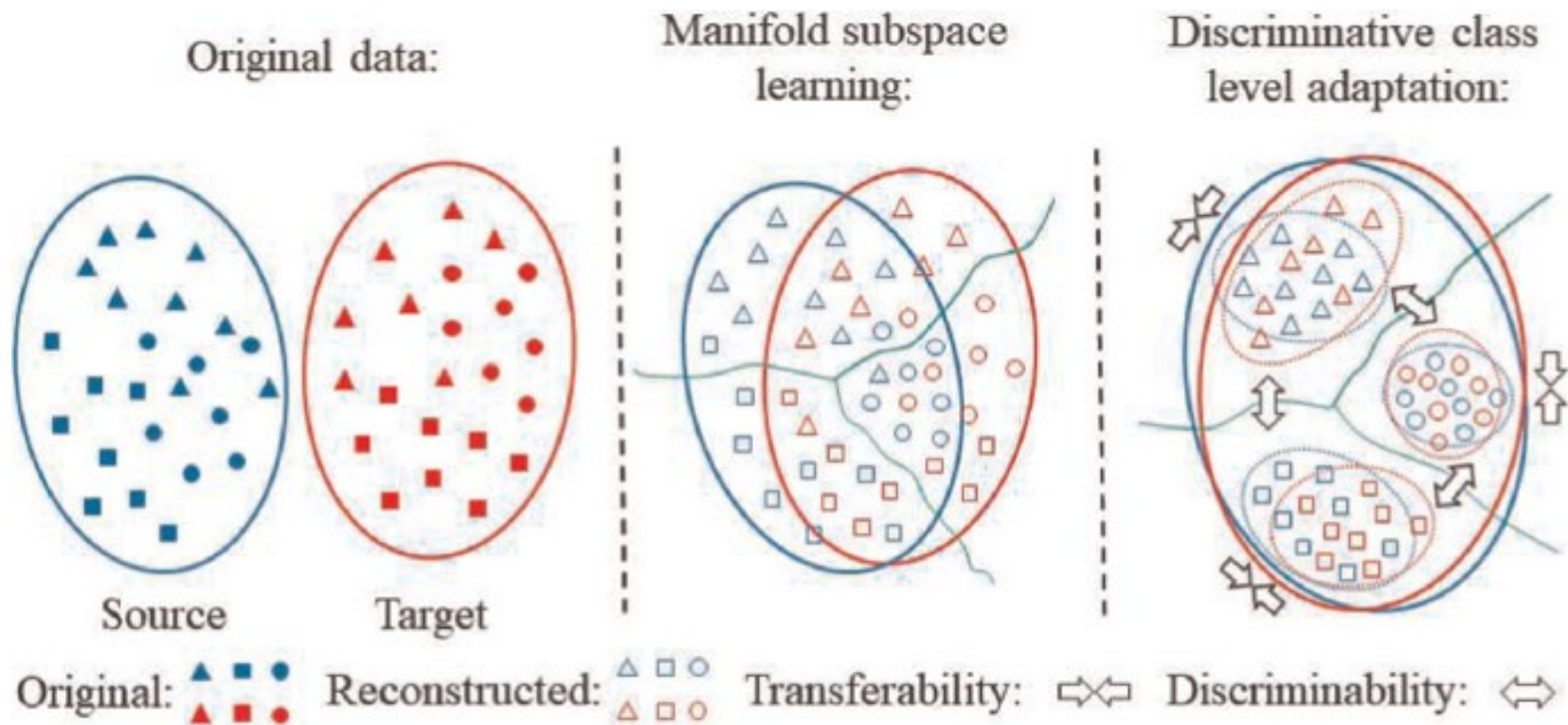
Variation



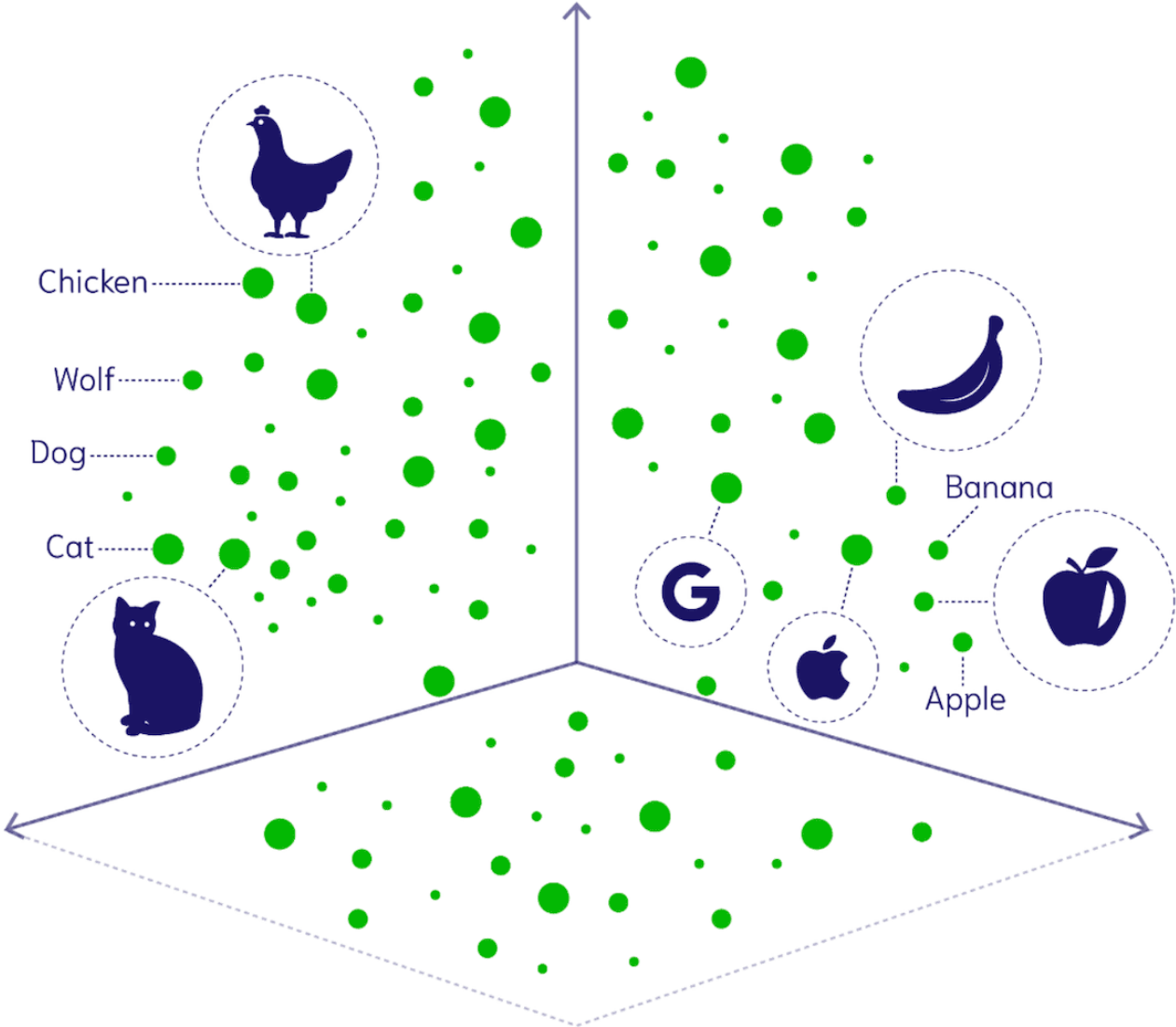
Variation



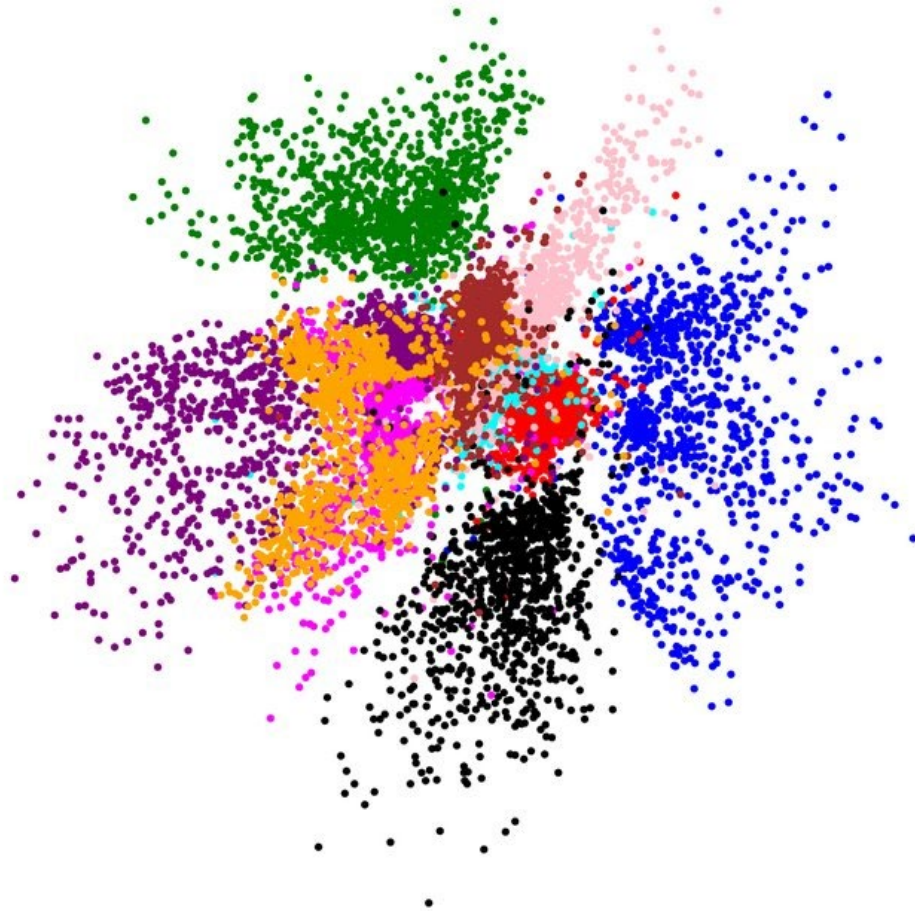
Manifold alignment



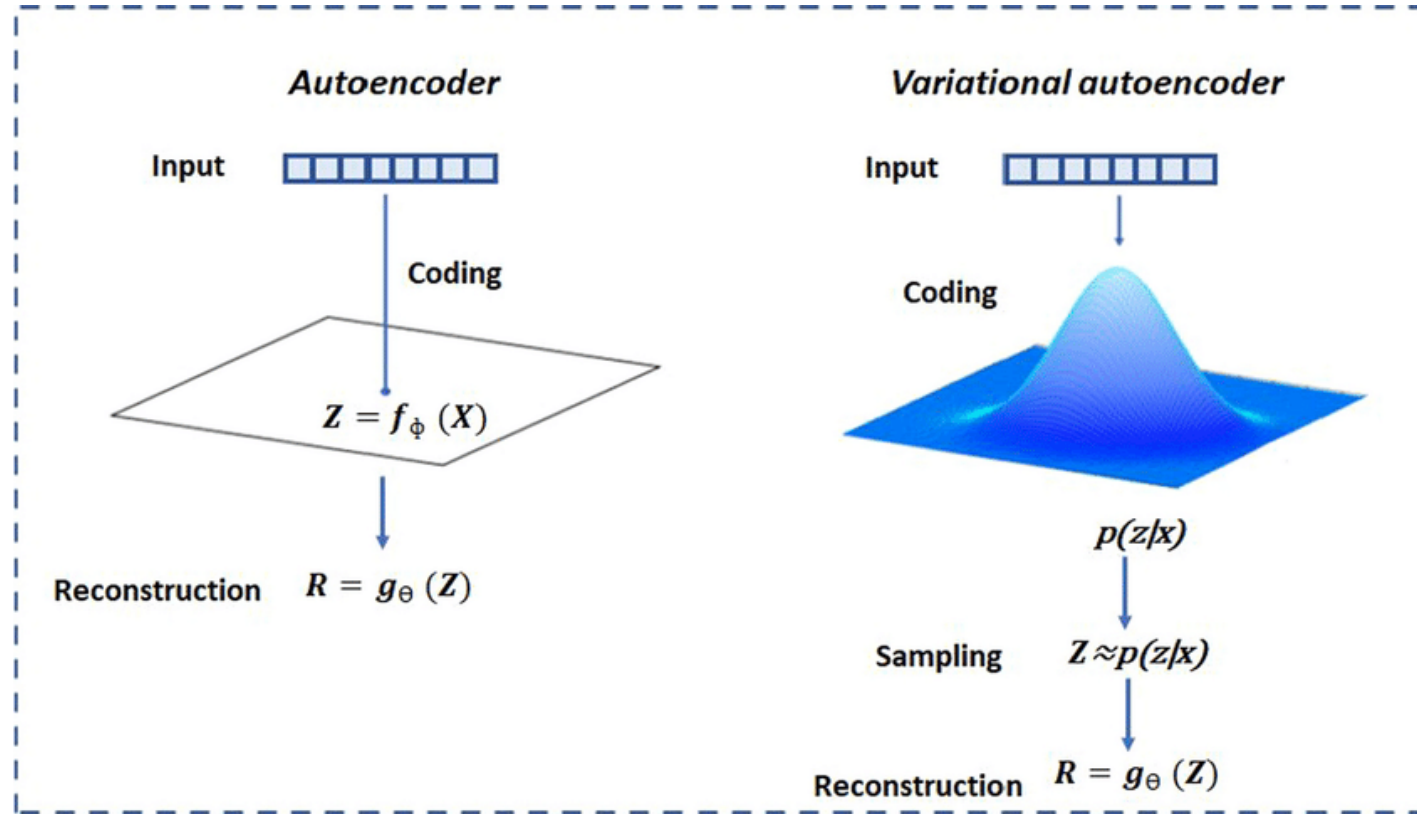
Multimodal spaces



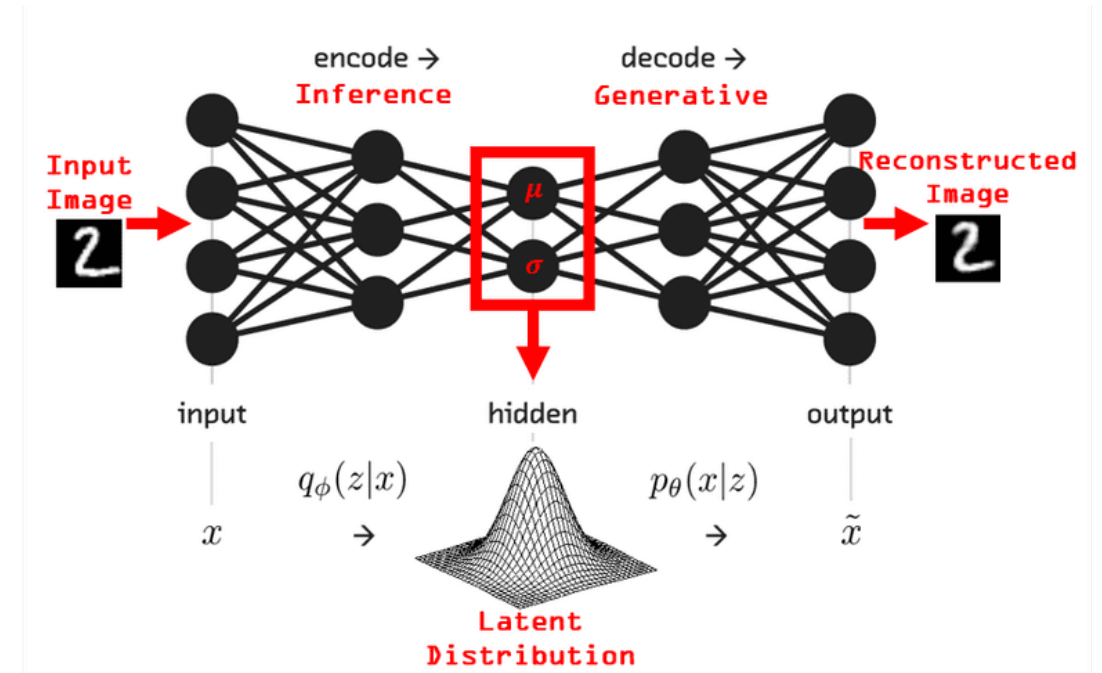
Shape of the latent space?



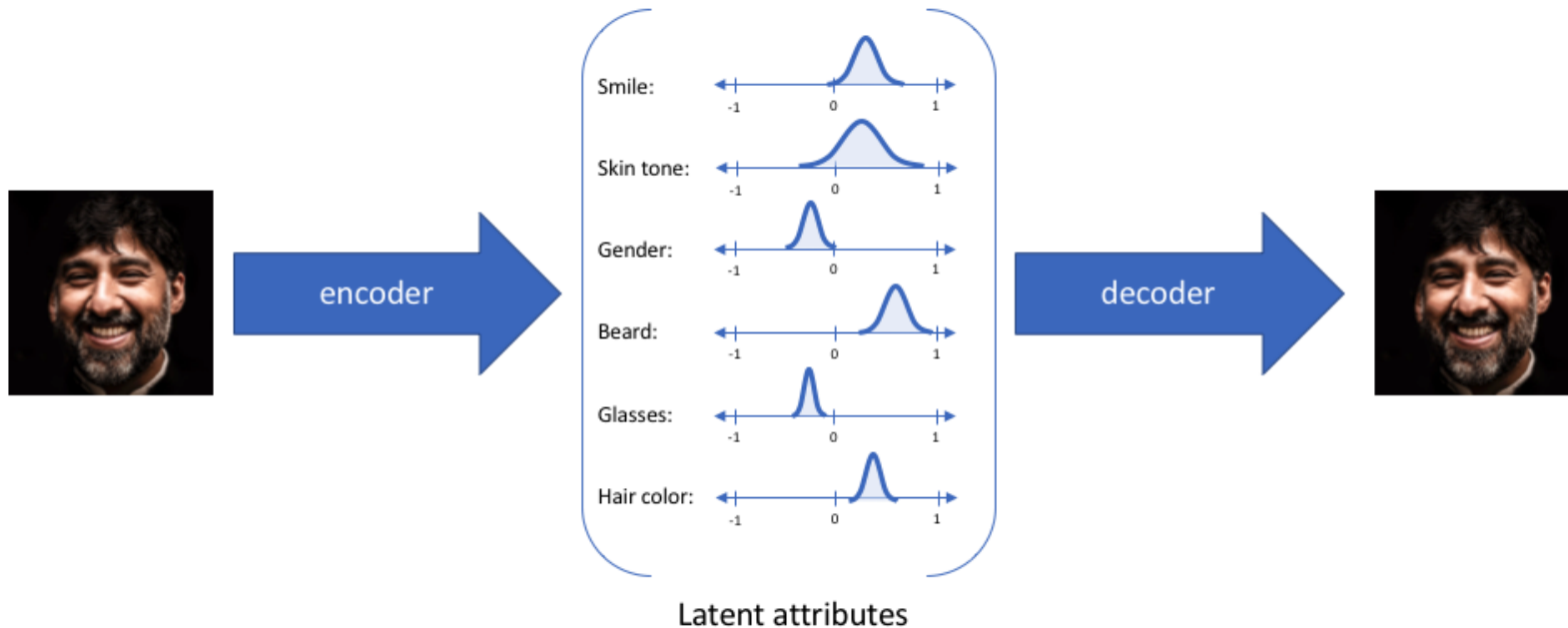
Central symmetry



Variational Autoencoders (VAE)

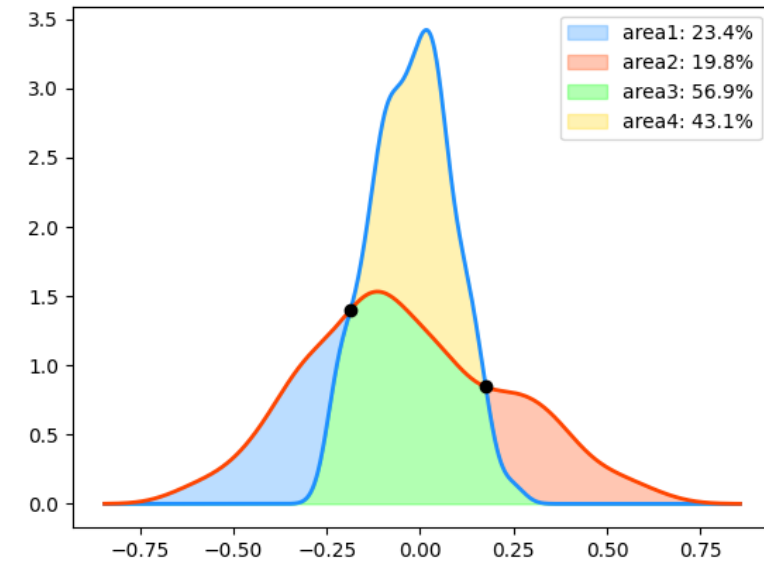


Impose meaning on the latent variables



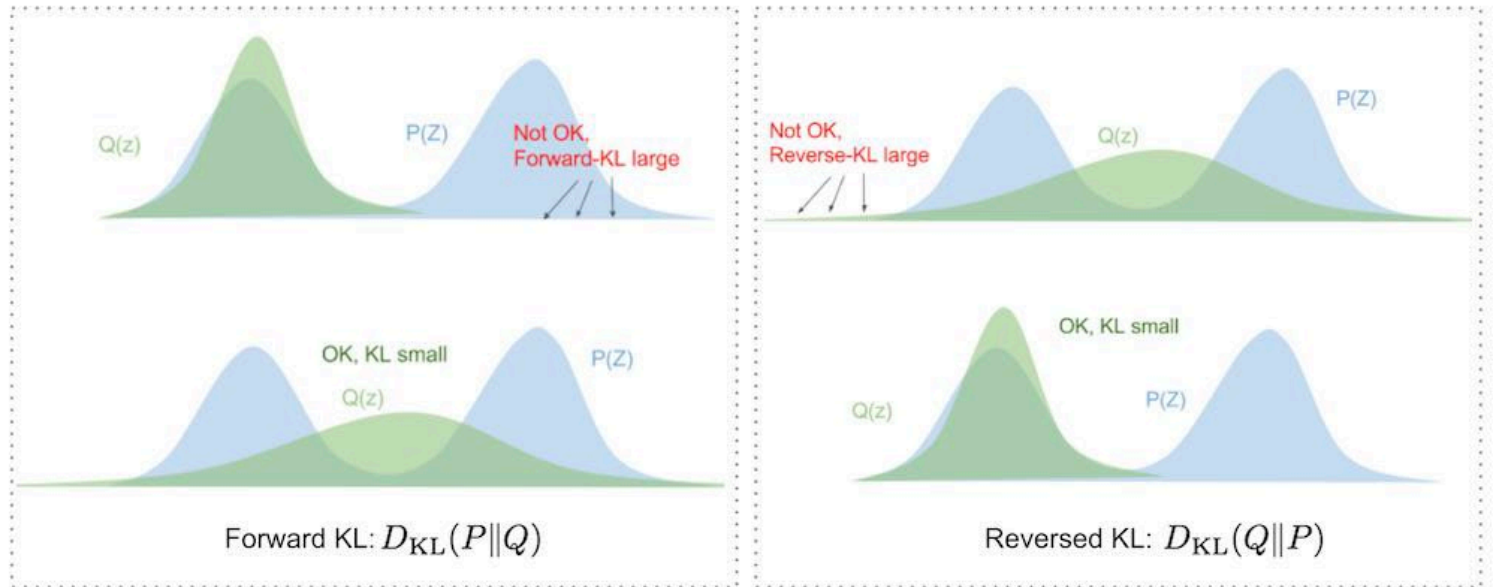
How to shape the latent space?

$$\mathcal{L}(x, \hat{x}) \rightarrow \mathcal{L}(x, \hat{x}) + \textit{Measure of distributional overlap}$$



Kullback-Leibler divergence

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}$$



VAE Loss

$$\mathcal{L}(x, \hat{x}) = MSE + KLD$$

$$KLD = \frac{1}{2} \sum_i [1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2]$$

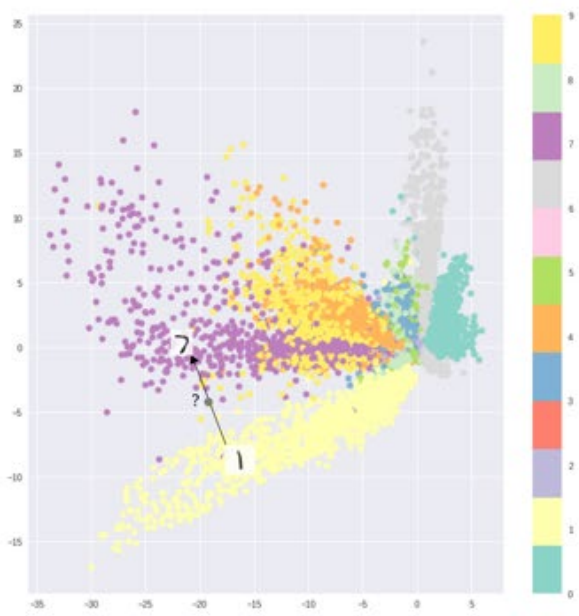
Mean and standard derivation of our
training sample embedding

(Target distribution is taken as implicit; mean = 0 and std = 1)

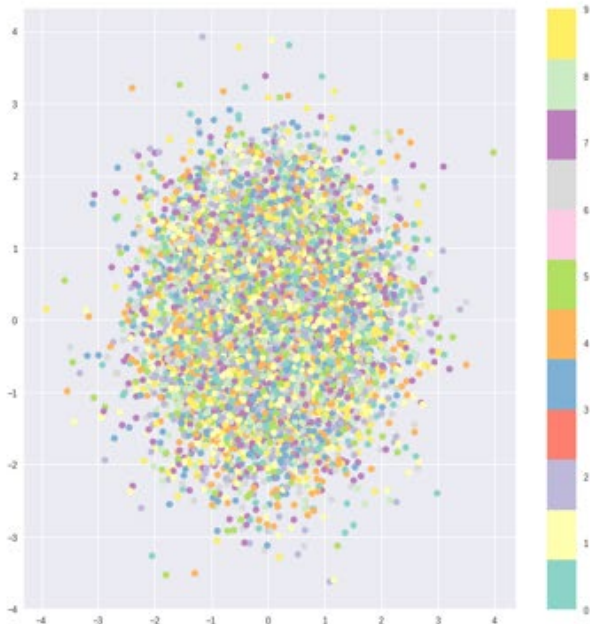
$$\begin{aligned} KL(p, q) &= \int [p(x) \log(p(x)) - p(x) \log(q(x))] dx \\ &= -\frac{1}{2} [1 + \log(2\pi\sigma_1^2)] - \left[-\frac{1}{2} \log(2\pi\sigma_2^2) - \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} \right] \\ &= \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} \end{aligned}$$

Effect:

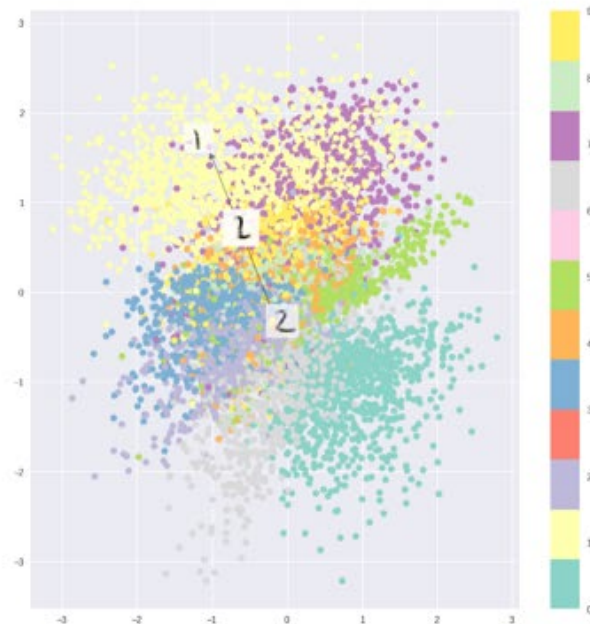
Only reconstruction loss



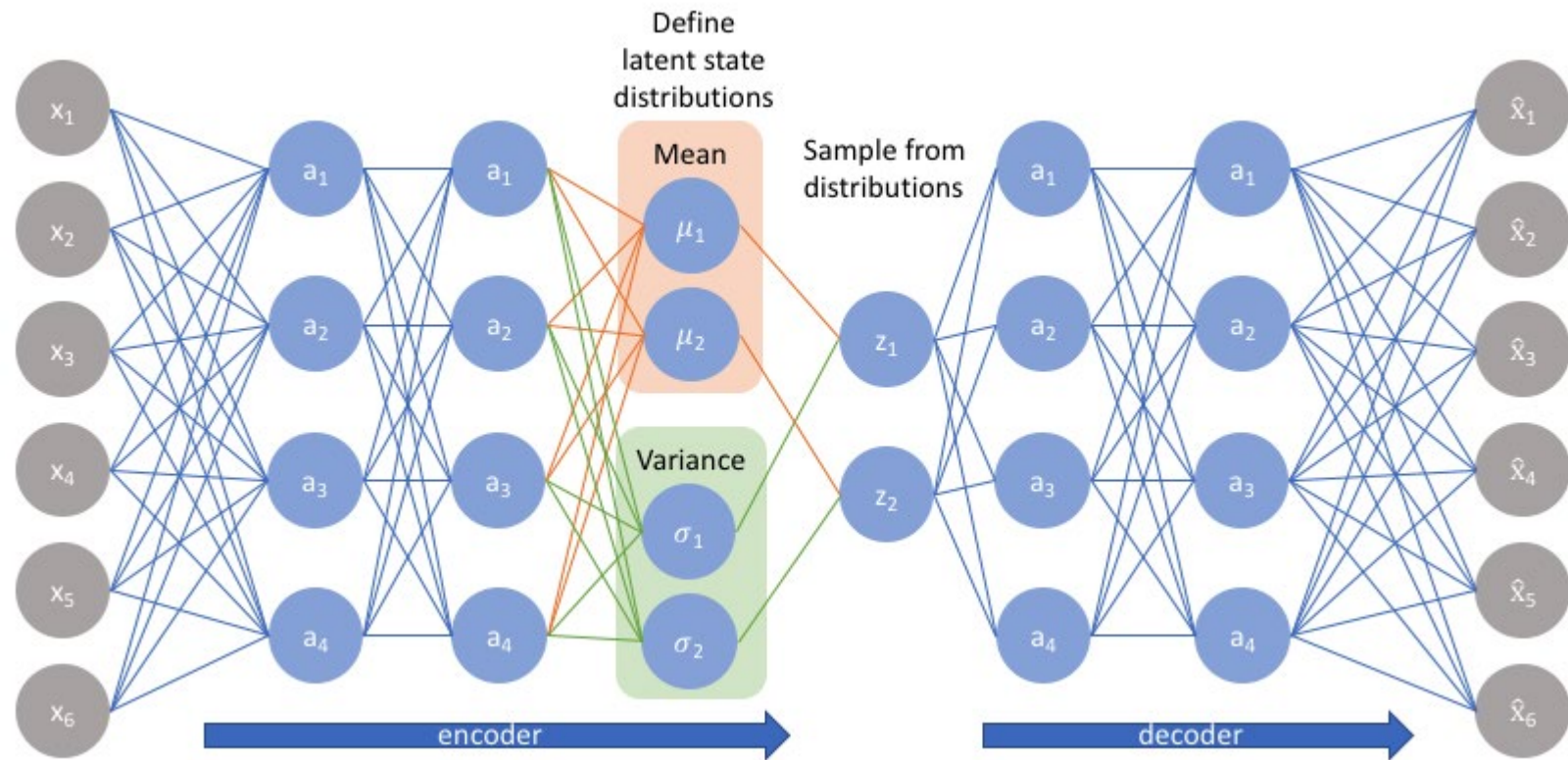
Only KL divergence



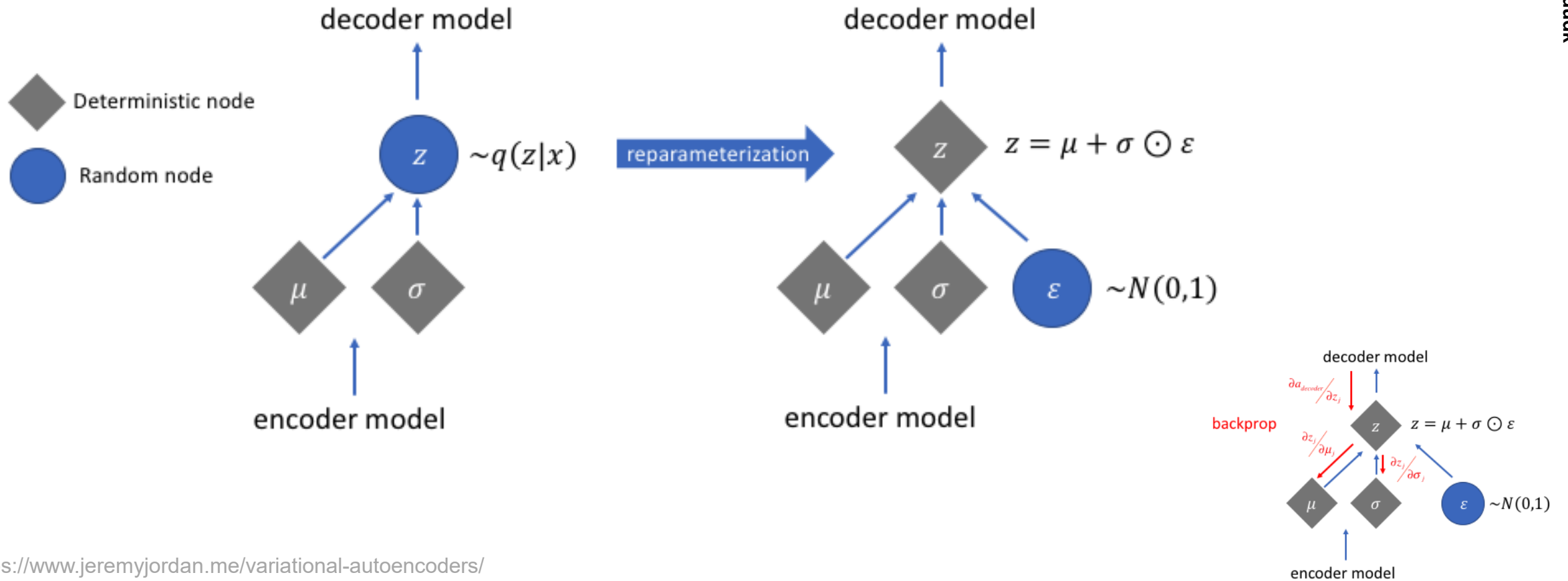
Combination



Architecture adjustments



Re-parametrisation



<https://www.jeremyjordan.me/variational-autoencoders/>

In code

```
class VAE(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(VAE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_size, latent_size),
            nn.LeakyReLU(True)
        )

        self.mean_layer = nn.Linear(latent_size, 2)
        self.logvar_layer = nn.Linear(latent_size, 2)

        self.decoder = nn.Sequential(
            nn.Linear(2, latent_size),
            nn.LeakyReLU(True),
            nn.Linear(latent_size, input_size)
        )

    def encode(self, x):
        x = self.encoder(x)
        mean = self.mean_layer(x)
        logvar = self.logvar_layer(x)
        return mean, logvar

    def reparameterization(self, mean, var):
        epsilon = torch.randn_like(var).to(device)
        z = mean + var * epsilon
        return z

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        mean, logvar = self.encode(x)
        z = self.reparameterization(mean, logvar)
        x_hat = self.decode(z)
        return x_hat, mean, logvar
```

sdu.dk

#sdu.dk

Compact variant

```
class VAE(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(VAE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_size, 128),
            nn.ReLU(True),
            nn.Linear(128, latent_size * 2) # mean and logvar
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_size, 128),
            nn.ReLU(True),
            nn.Linear(128, input_size),
            nn.Sigmoid()
        )

    def encode(self, x):
        x = self.encoder(x)
        mean = x[:, :latent_size]
        logvar = x[:, latent_size:]
        return mean, logvar

    def reparameterization(self, mean, var):
        epsilon = torch.randn_like(var).to(mean.device)
        z = mean + var * epsilon
        return z

    def decode(self, x):
        return self.decoder(x)

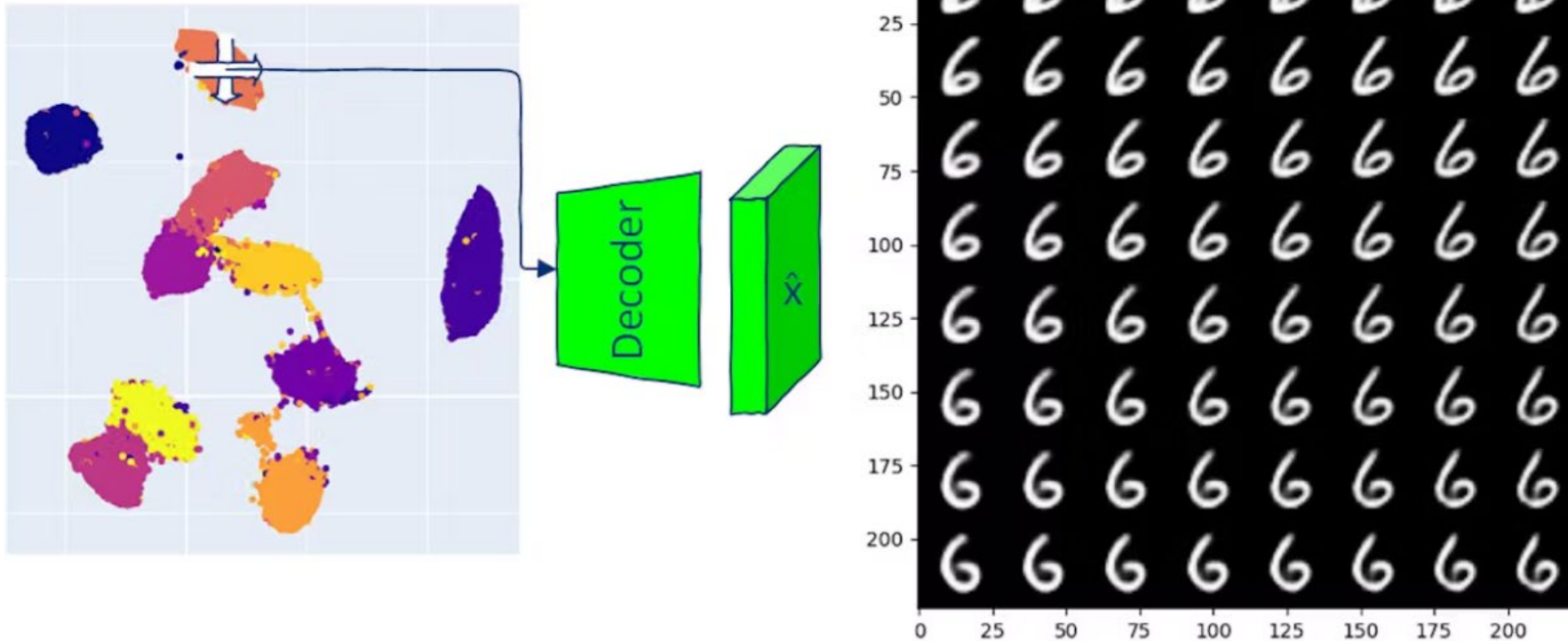
    def forward(self, x):
        mean, logvar = self.encode(x)
        z = self.reparameterization(mean, logvar)
        x_hat = self.decode(z)
        return x_hat, mean, logvar
```


If you have labels, use them!

⇒ Conditioning

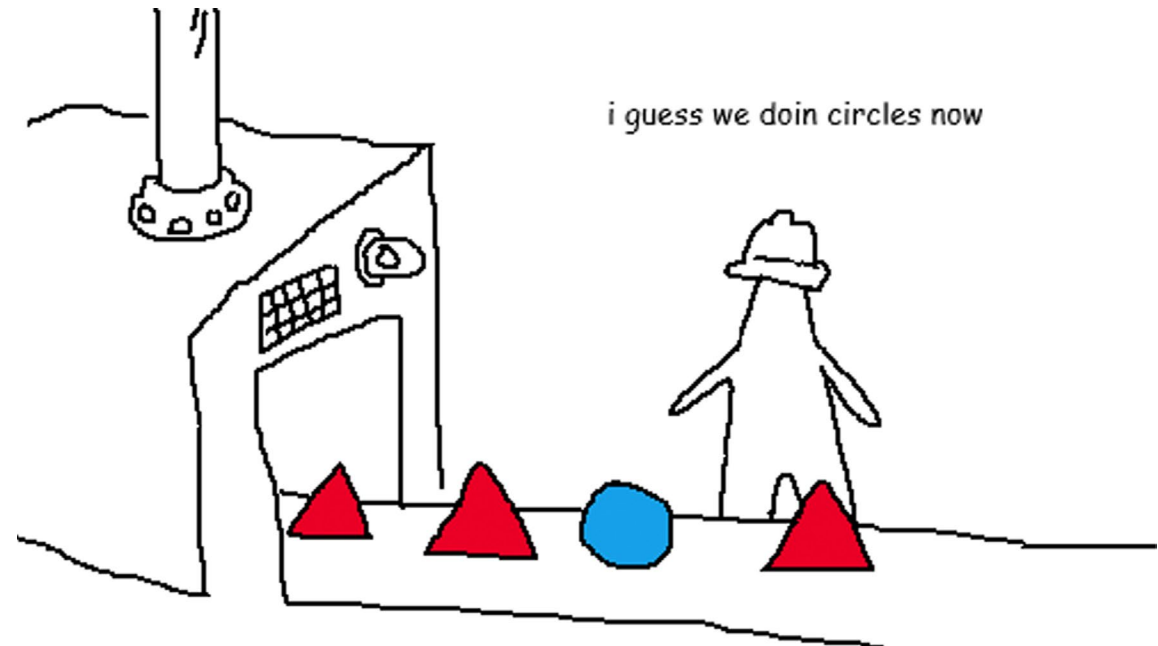


Sample from gaussians



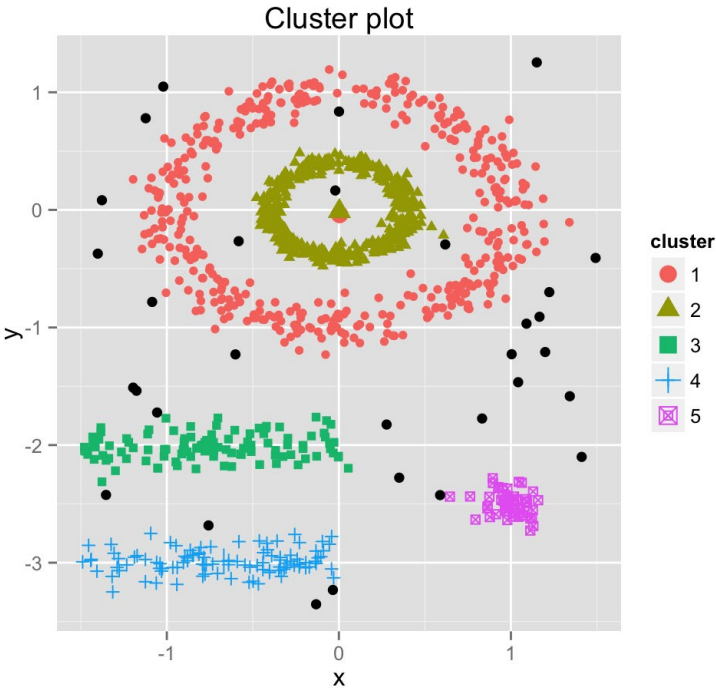
<https://hackernoon.com/how-to-sample-from-latent-space-with-variational-autoencoder>

Case example: Anomaly Detection

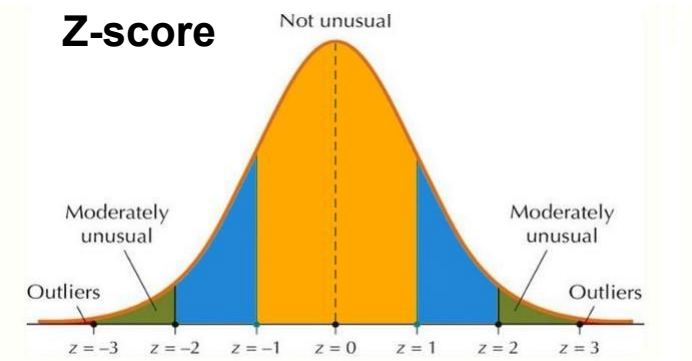
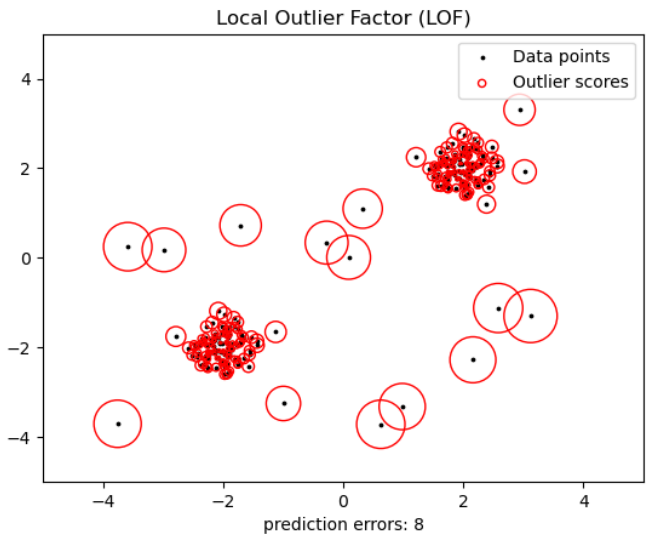


Anomaly detection

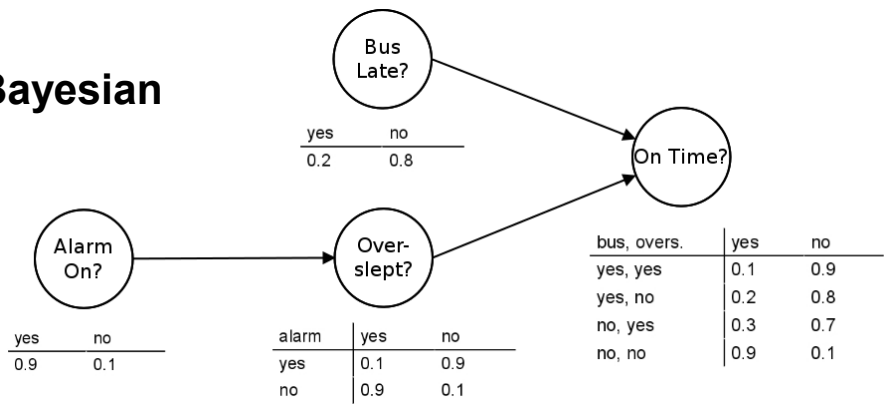
Cluster based



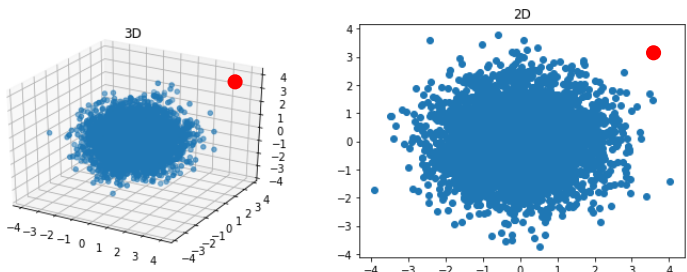
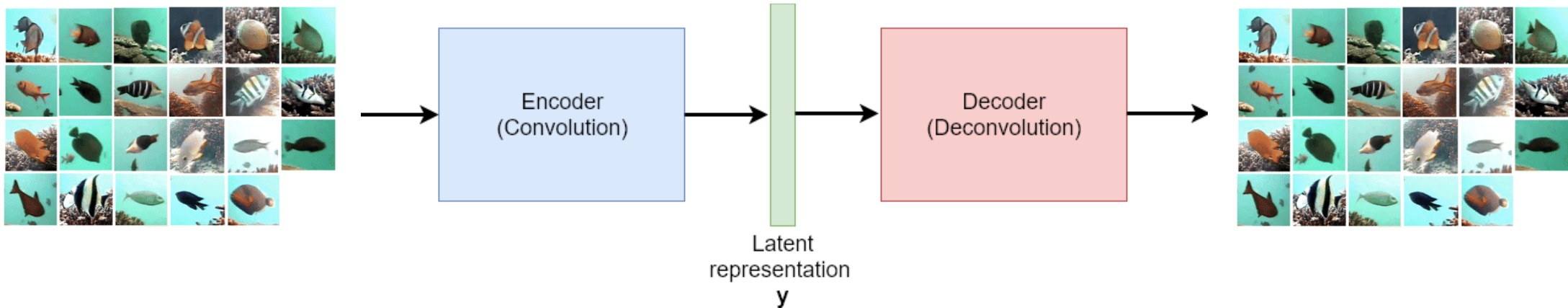
Density based



Bayesian



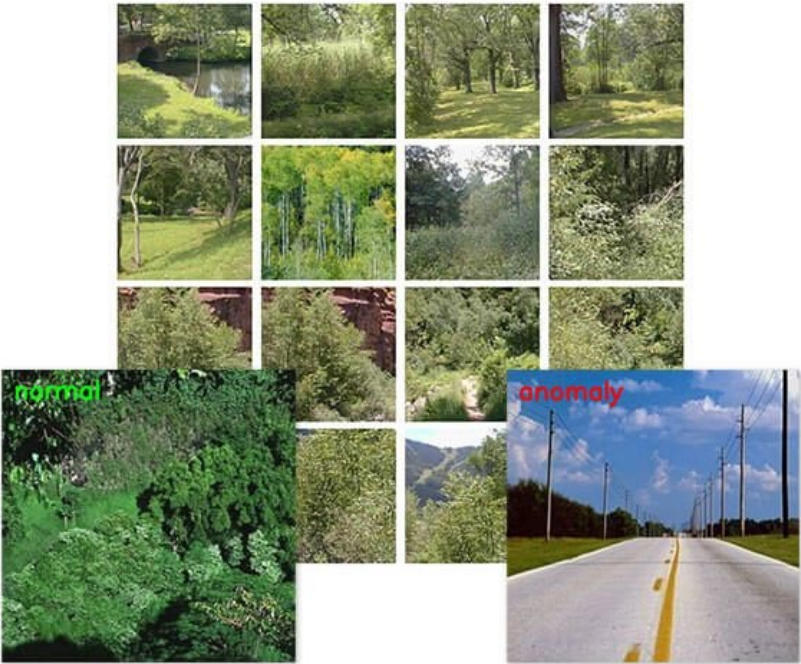
Reconstruction based anomalies



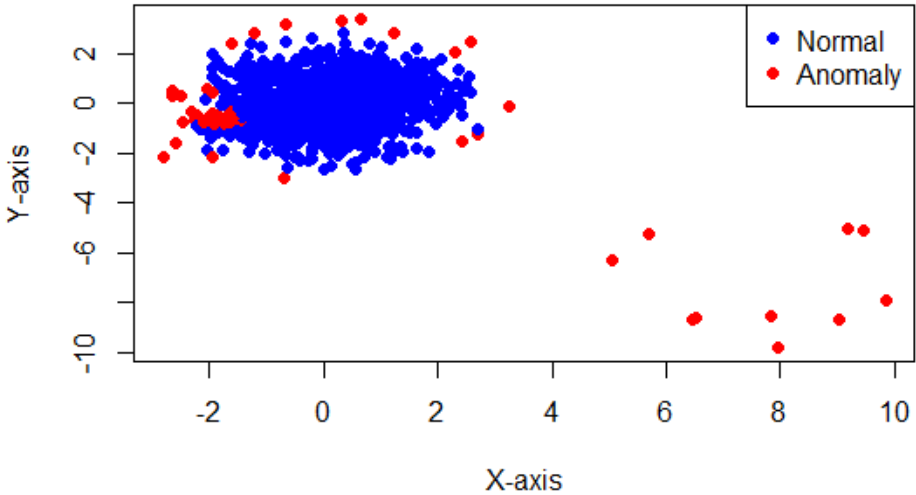
$$E(\text{rec_loss}) \ll \text{anomalous rec_loss}$$

Unsupervised anomaly detection

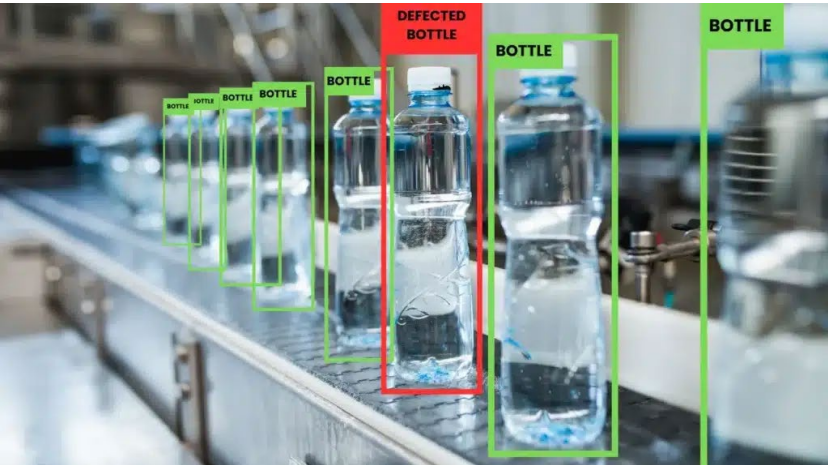
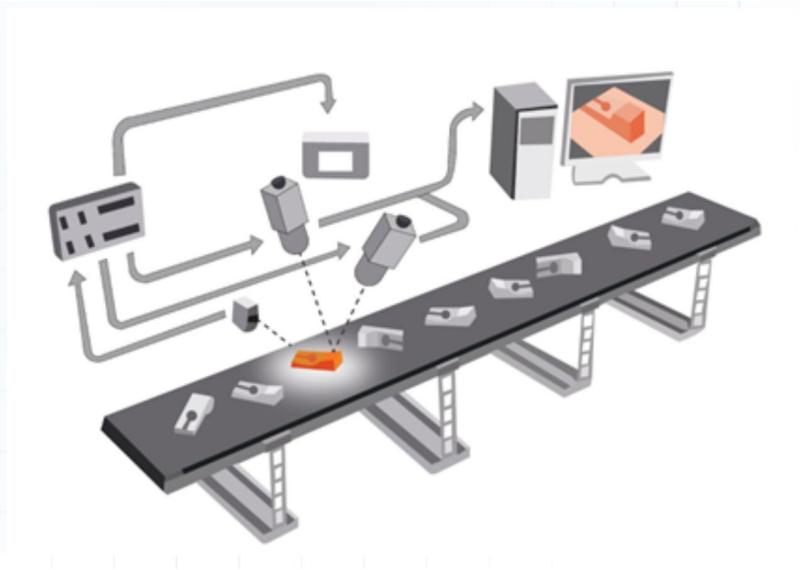
Forest Dataset



Autoencoder Anomaly Detection



Practical case



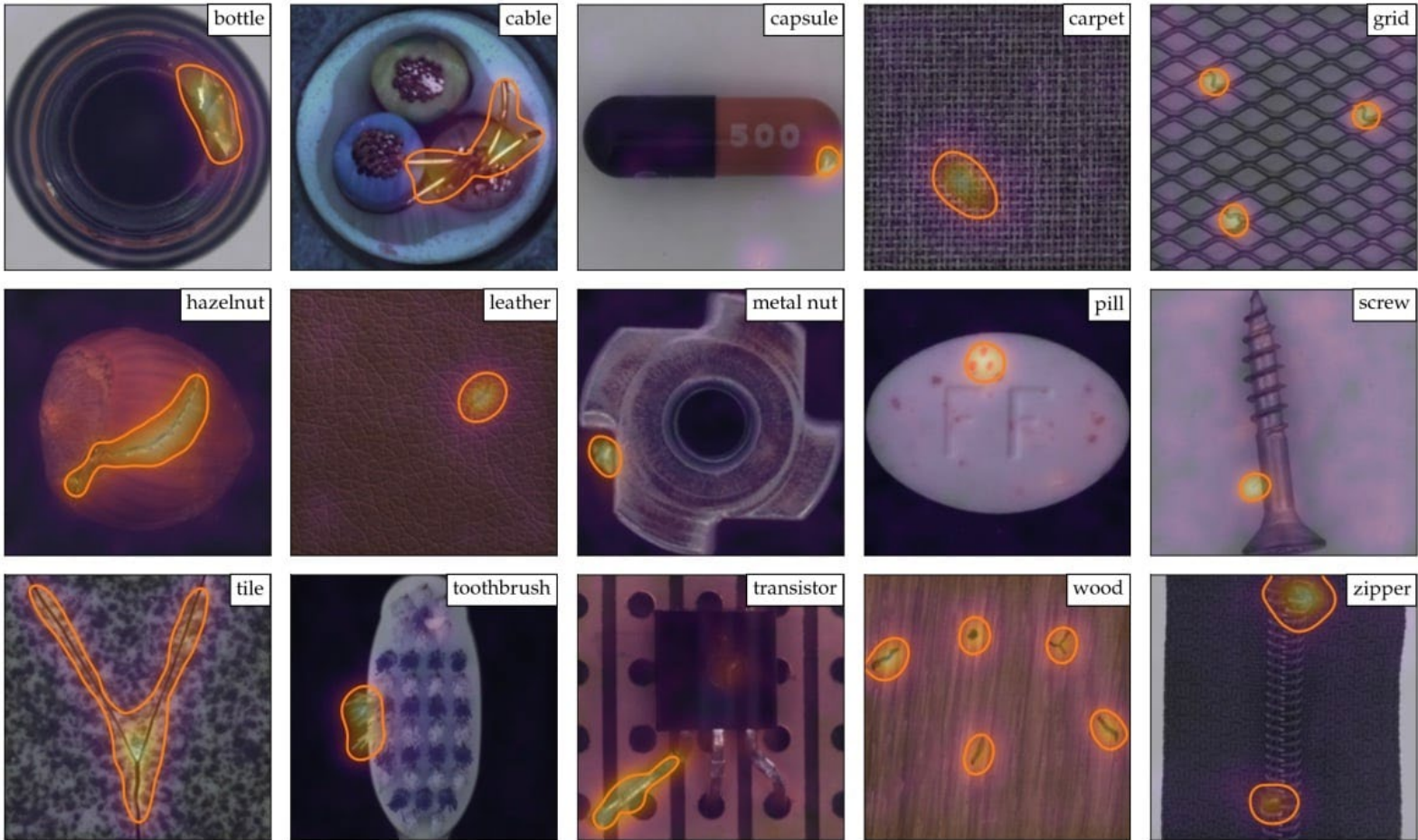
Images without anomalies



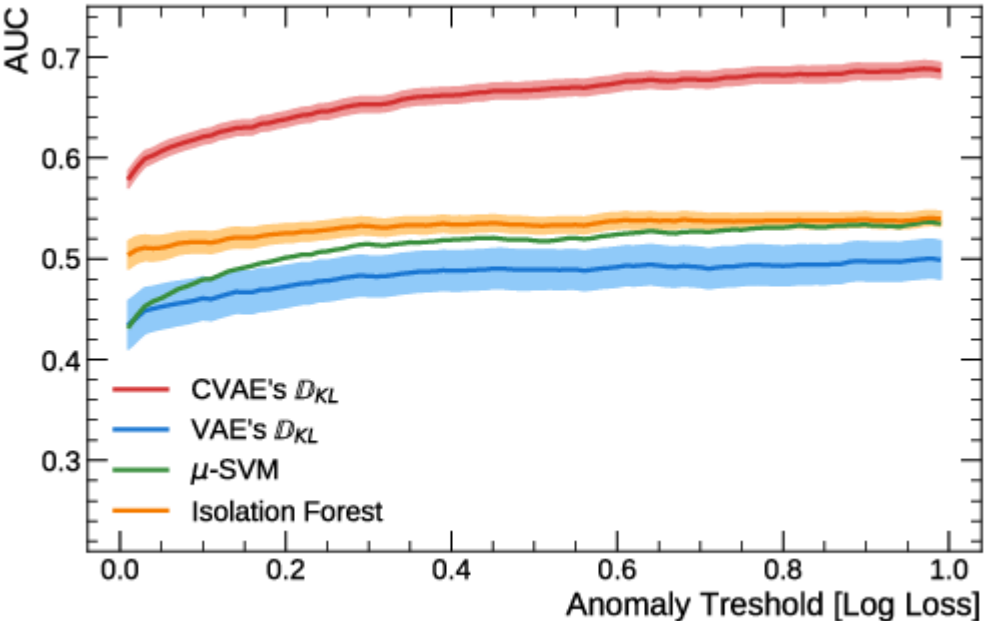
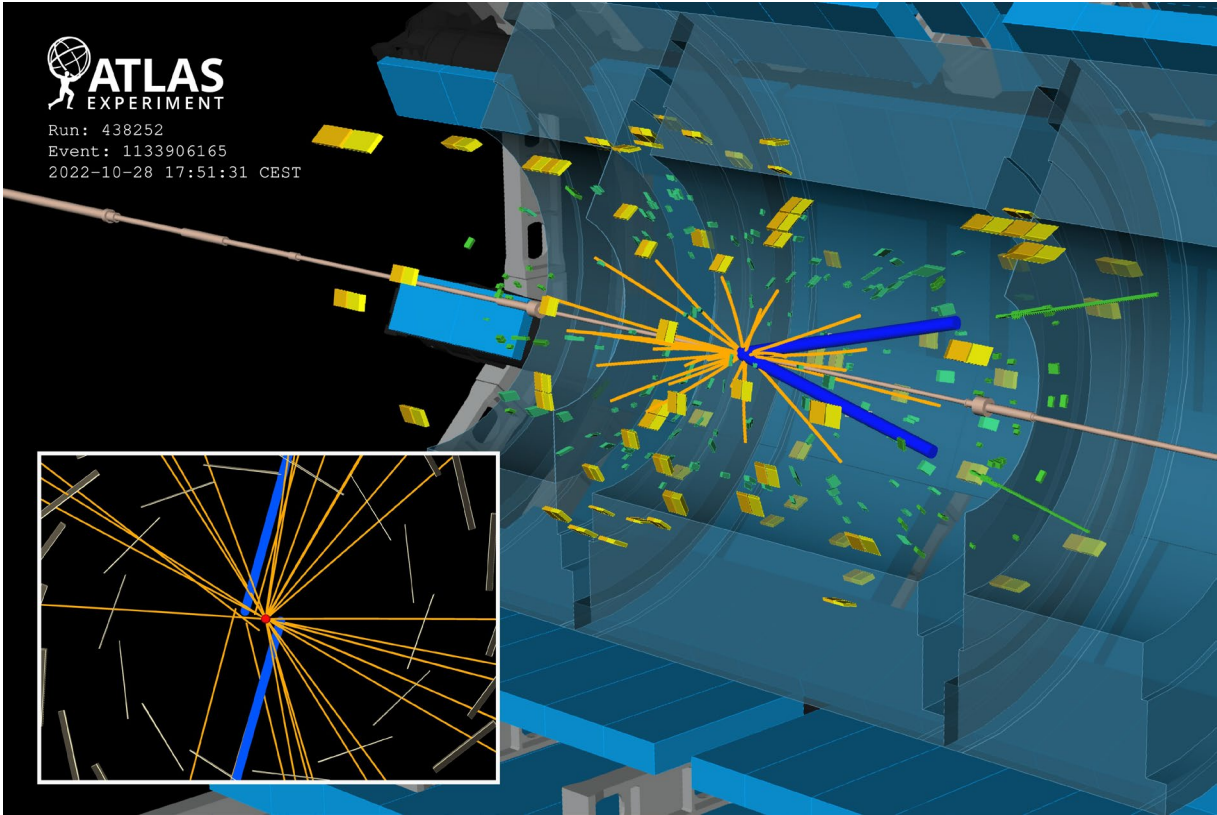
Images with anomalies



Anomaly + explanations



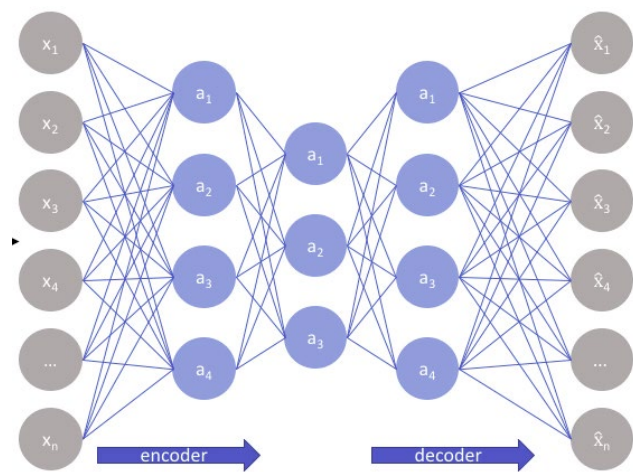
Practical case



Case example: Interpretability with Sparse Autoencoders



Compression vs sparsity



Department of Mathematics
and Computer Science

What is the fuss about?



ID	Height	Age	SP	Colour	Pet
1	190	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps ...

Encoder

'Bottleneck'

Decoder



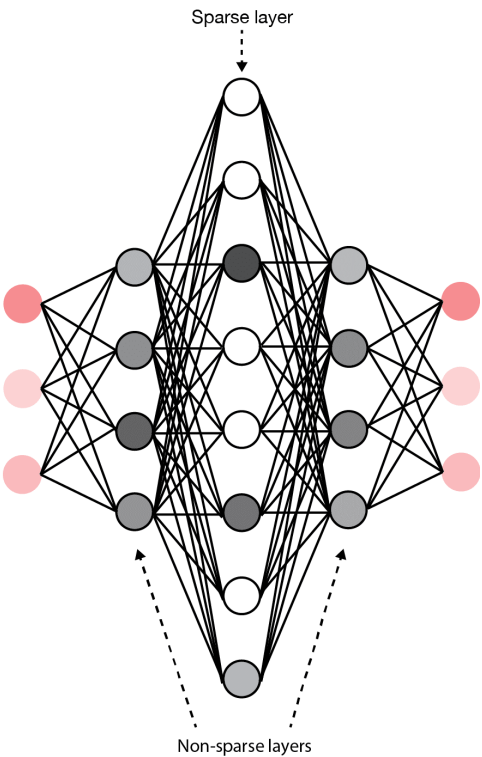
ID	Height	Age	SP	Colour	Pet
1	190	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps ...

SDU

Anton D. Lautrup ~ lautrup@imada.sdu.dk

23 October 2025

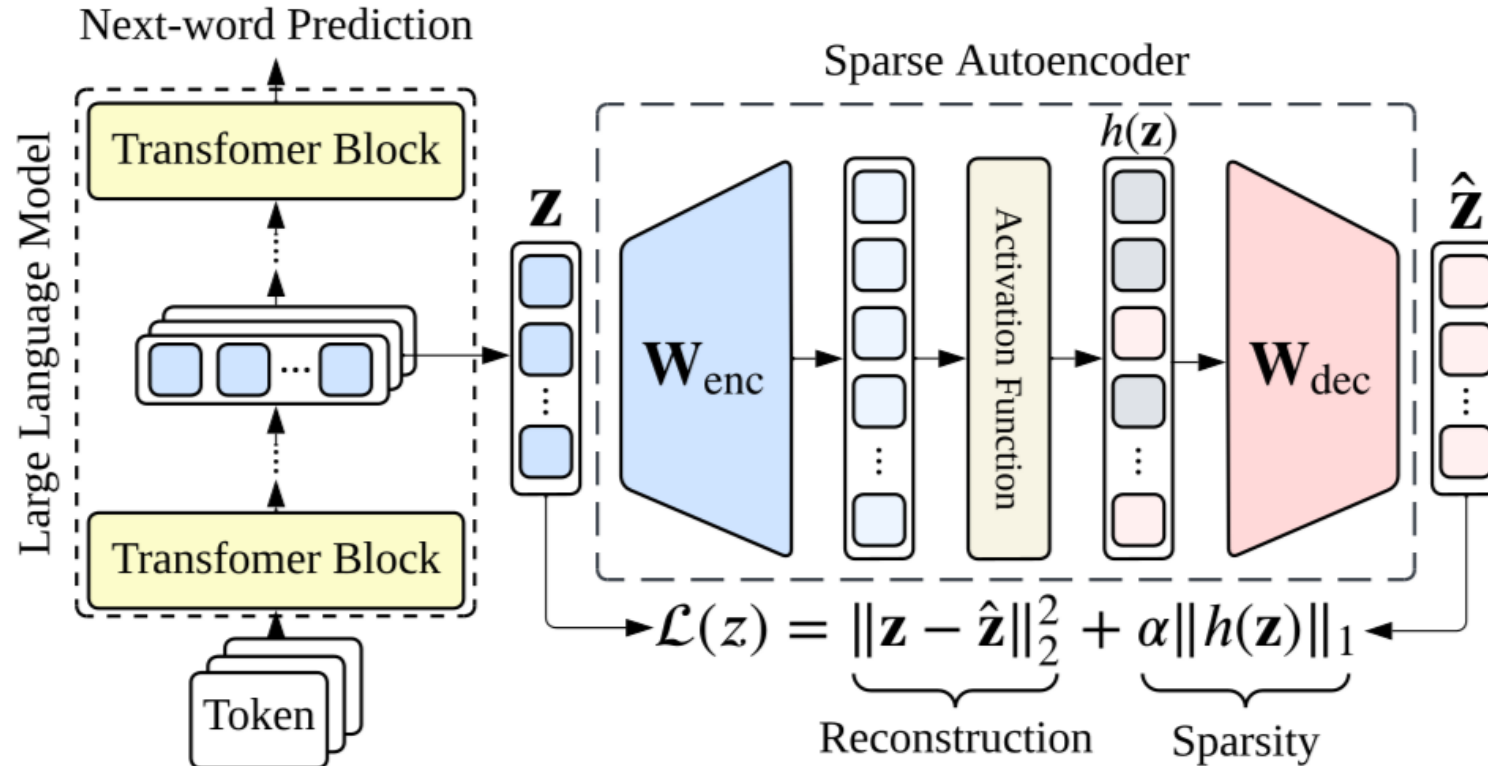


Sparsity constraint

→ Build an autoencoder that can disentangle rich semantic model states into human interpretable features (higher-dimensional but sparse).

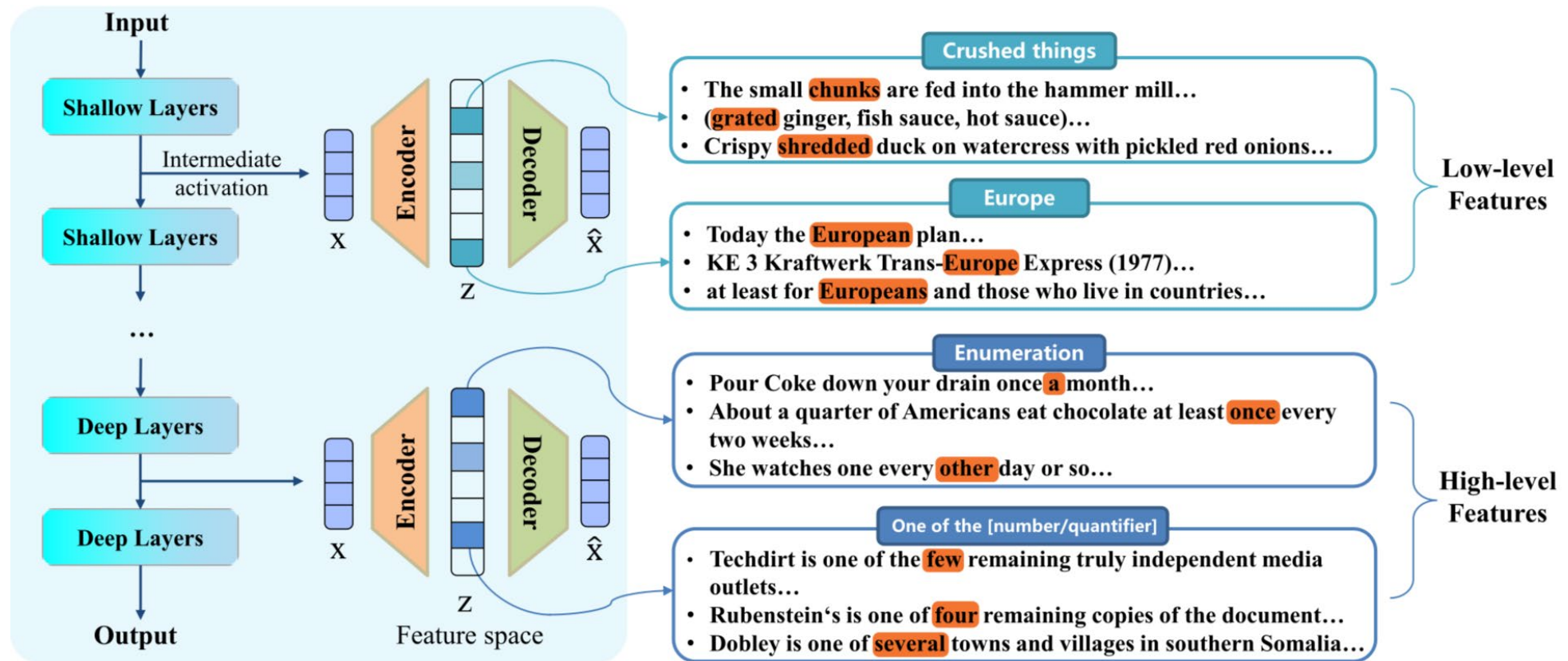
$$\mathcal{L}(x, z) = \underbrace{\frac{1}{2} \|g(f(x)) - x\|^2}_{\text{(regular rec. loss)}} + \underbrace{\alpha \|z\|}_{\substack{\text{(strength parameter)} \\ \text{"L1 regularisation" on the embedding}}}$$

LLM Explanations



(a) SAE Framework

LLM Explanations

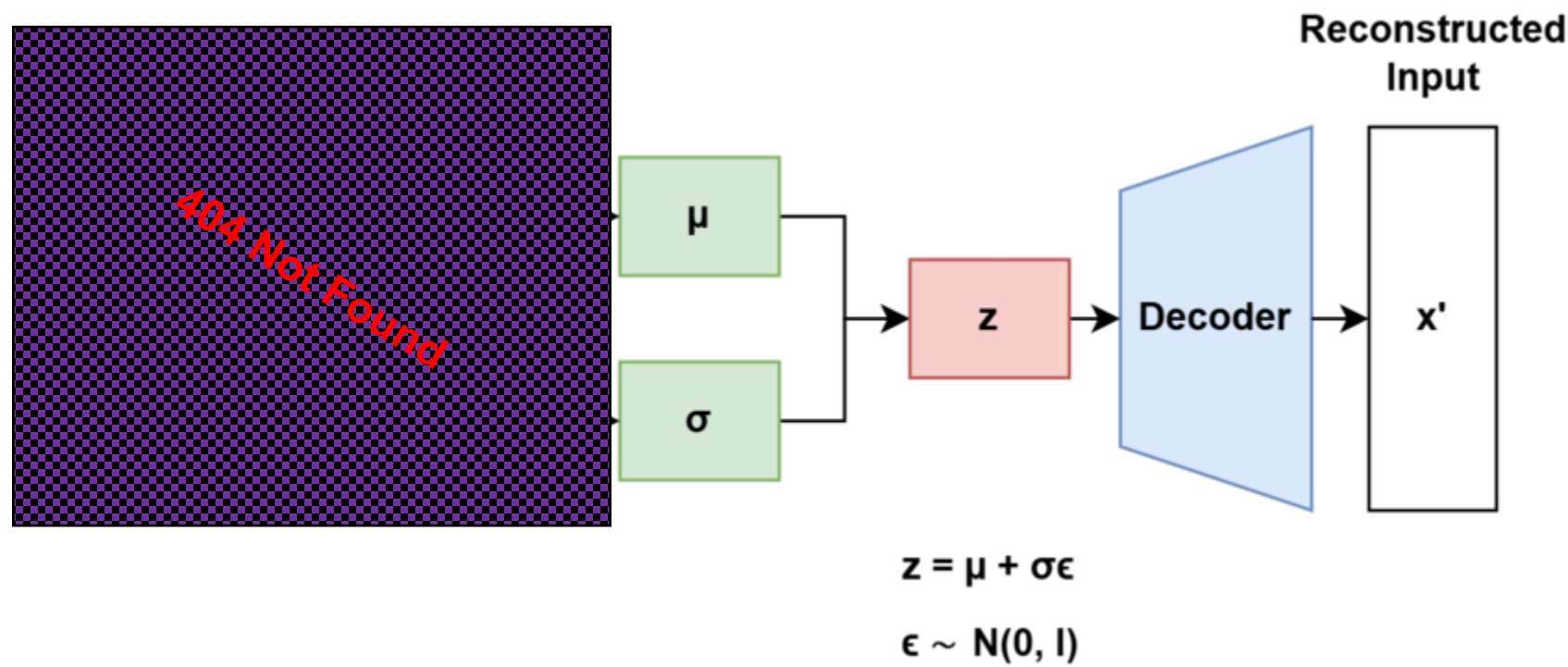


Case example:

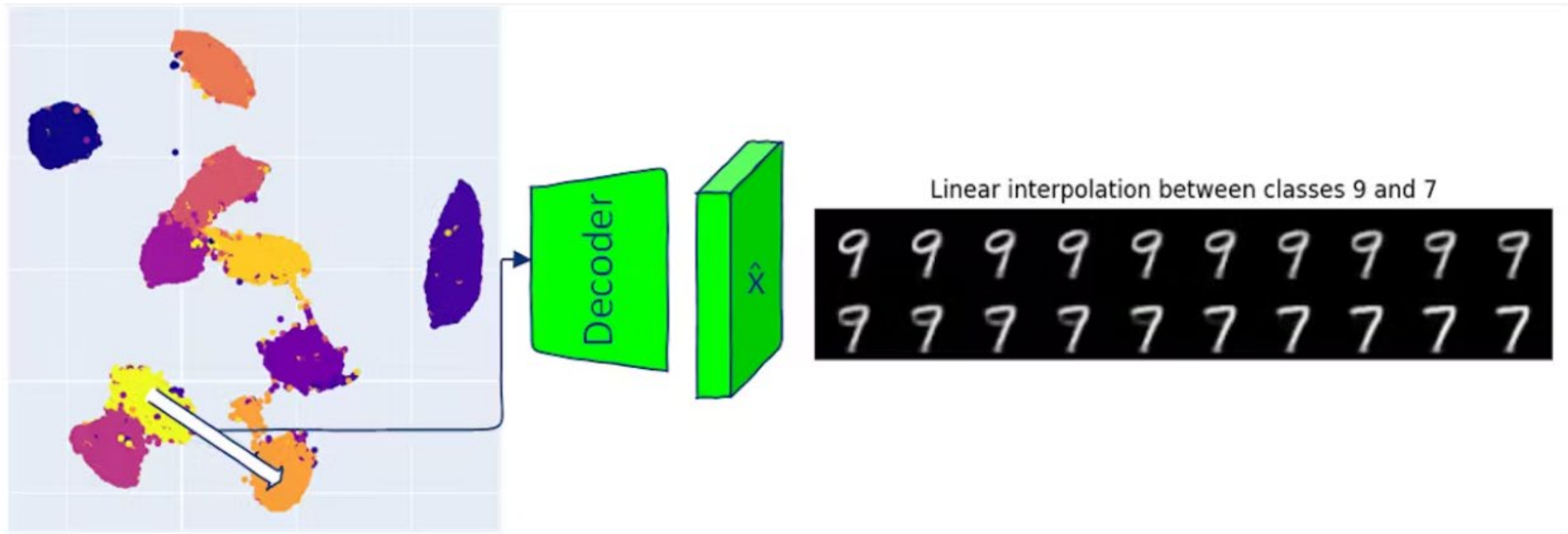
Generative Model



Generative VAE

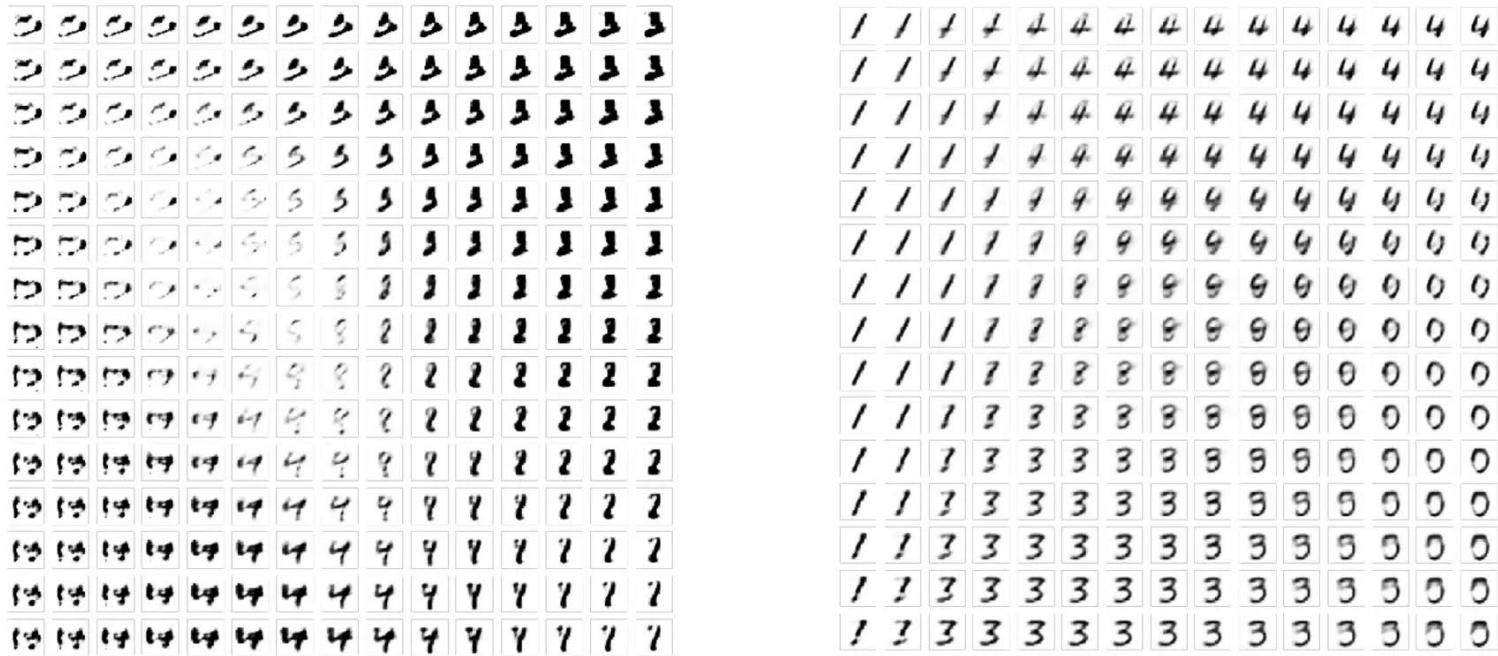


Not a very far-fetched next step



AE vs VAE digit generation

left = AE generated digits, right = VAE generation digits (10-D latent space).



Blessing/Curse of dimensionality

EXAMPLE 1 Consider how many *almost orthogonal* unit vectors we can have in space, such that all pairwise angles lie between 88 degrees and 92 degrees.

In $\mathbb{R}^2$ the answer is 2. In $\mathbb{R}^3$ it is 3. (Prove these to yourself.)

In $\mathbb{R}^d$ the answer is $\exp(cd)$ where $c > 0$ is some constant.

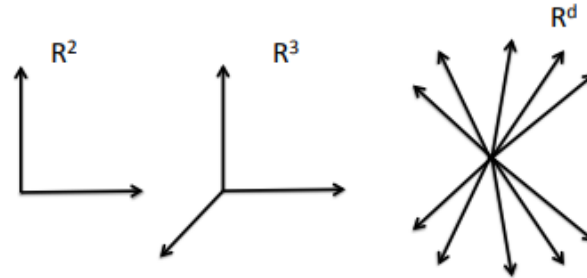
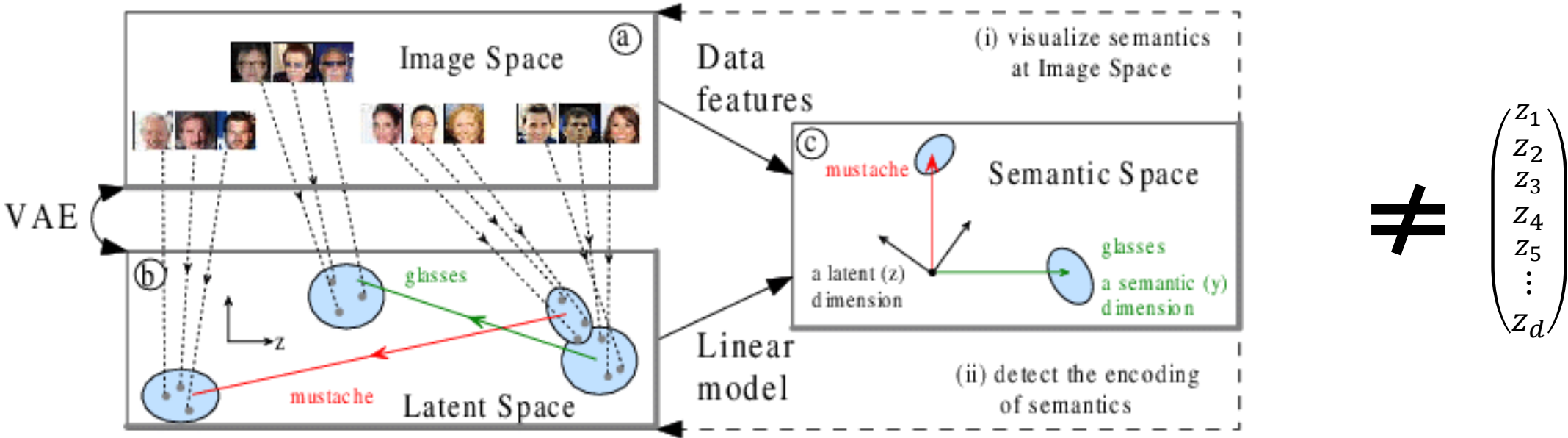


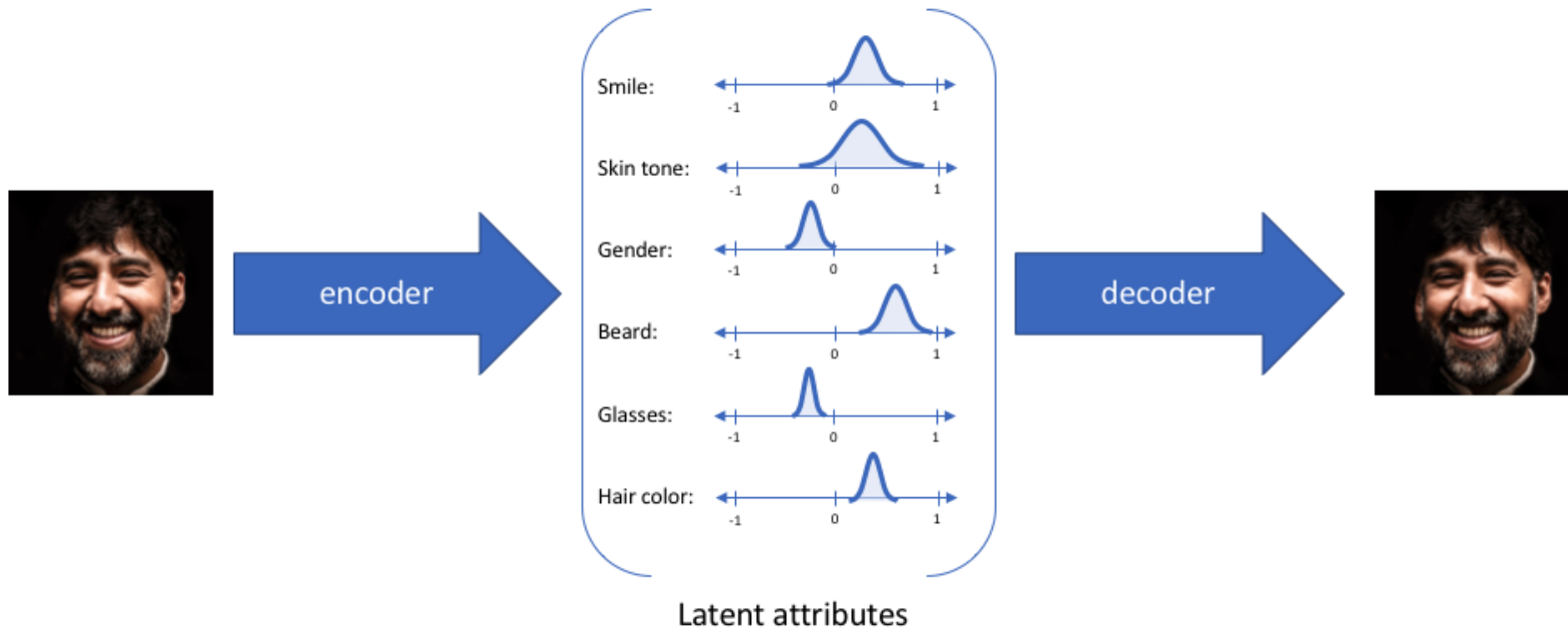
Figure 1: Number of almost-orthogonal vectors in $\mathbb{R}^2, \mathbb{R}^3, \mathbb{R}^d$

Semantic vs. specific dimensions

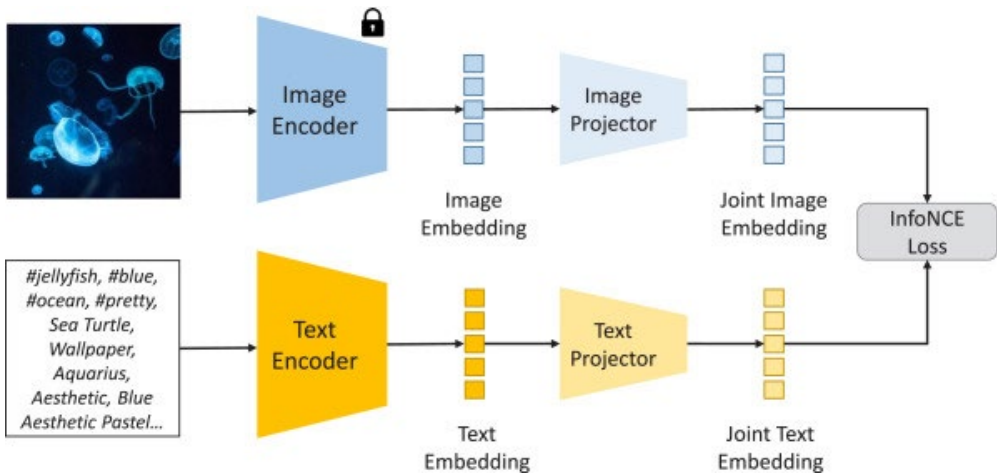
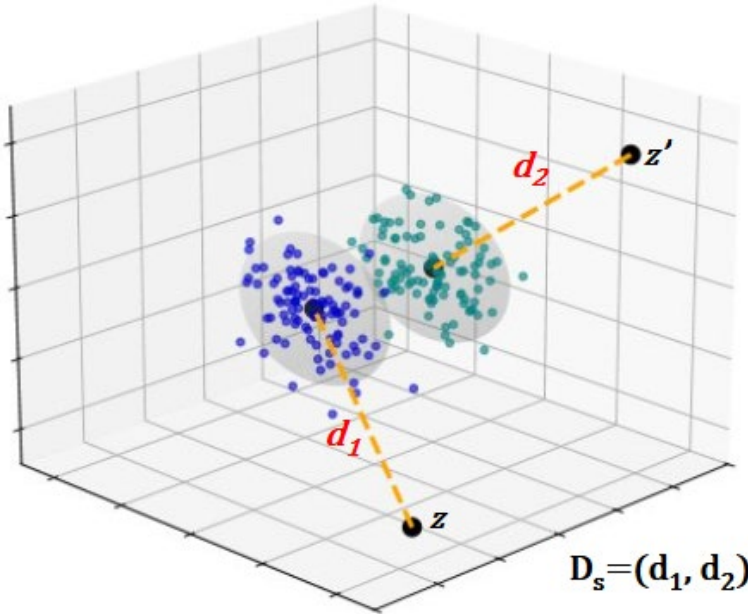
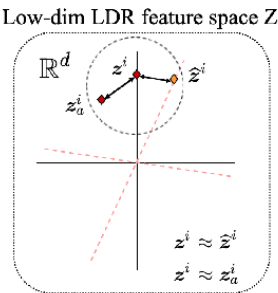
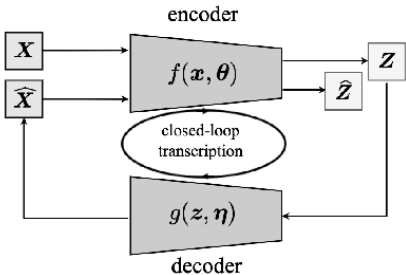
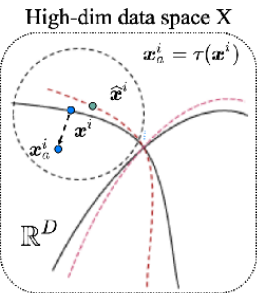


LatentVis: Investigating and Comparing Variational
Auto-Encoders via Their Latent Space. <https://ceur-ws.org/Vol-2699/paper04.pdf>

Impose meaning on the latent variables

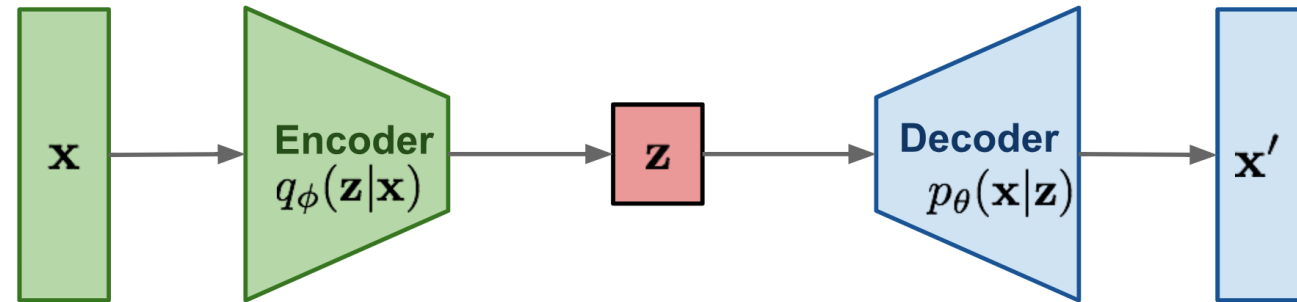


Everything is embeddings

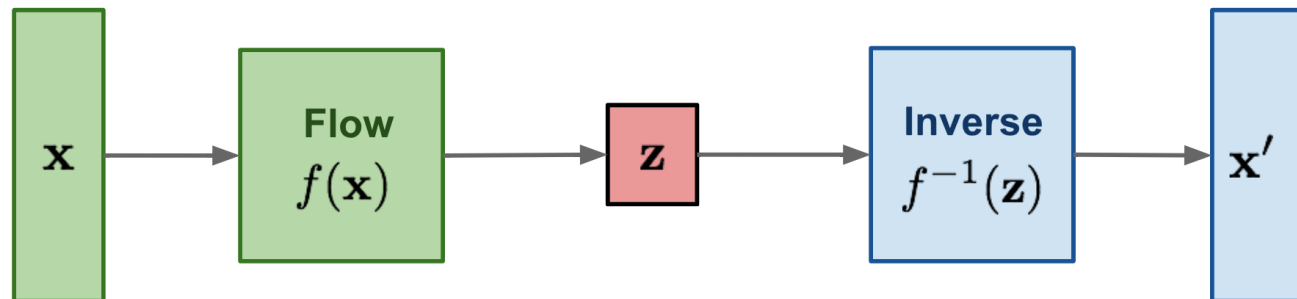


Flow Models: Just to let you know it exists

VAE: maximize ELBO.



**Flow-based
generative models:**
minimize the negative
log-likelihood



<https://arxiv.org/pdf/1908.09257>

Questions?

